

DATA STRUCTURES AND ALGORITHMS
23CSE203 LAB MANUAL

SUBMITTED BY	
NAME	TANAY CHIRUMAMILLA
ROLL NO	AV.SC.U4CSE25150
YEAR/SEM/SECTION	2nd YEAR/SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST. PROFESSOR (SI. GRADE)
DEPARTMENT	SCHOOL OF COMPUTING

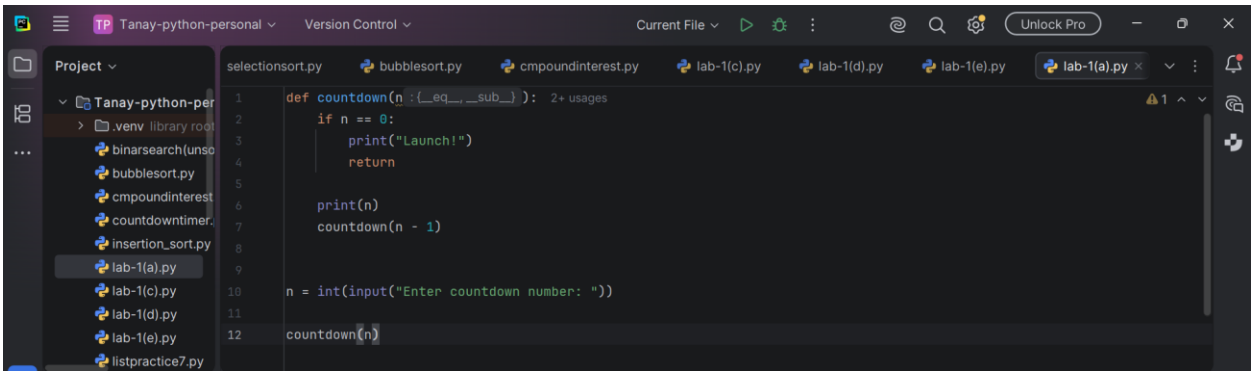
S.NO	Title	Page no	Signature
1	Week1 (a). A rocket launch system displays a countdown before launch. Instead of loop implement countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!". (b) Banks calculate the growth of an investment using compound interest. The formulae require calculating powers, which can be implemented recursively. Write a recursive program to calculate P^N where: P=Principal Growth, N=number of years. (c) An HR department stores ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in the list. (d) In logistics company, develop a recursive program to calculate the factorial of a given number. (e) Write a recursive program to print the first n terms of the Fibonacci series	6-14	
2	Week-2 (a) Linear Search (b) Binary Search without sorting (c) Binary Search with sorted array	14-19	
3	Week-3 (a) Bubble Sort (b) Insertion Sort (c) Selection Sort	19-28	
4	Week 4 (a) Write a Python program to implement Merge Sort.	28-31	

	(b) Write a Python program to implement Quick Sort.		
5	Week 5 5(a). Write a Python program to implement a Singly Linked List (SLL) with the following operations: • Insert at beginning • Insert at end • Insert at a specific position • Delete by value • Delete the last node • Count the number of nodes • Display/Traverse • Exit 5(b) Write a Python program to implement a Double Linked List with basic operations. 5(c) Write a Python program to implement a Circular Linked List with basic operations.	32-40	
6	Week 6 Write a Python program to implement Stack operations such as Push, Pop, Peek, and Display using an array and the linked list.	40-48	
7	Week 7 (a) Write a Python program to implement a Queue and its **o**perations such as Enqueue, Dequeue, Peek, and Display with a menu-driven approach. Implement using array and SLL**.** (b) Write a Python Program to implement Circular Queue	48-52	
8		52-55	
9		55-58	
10		58-66	
11		67-74	
		74-77	

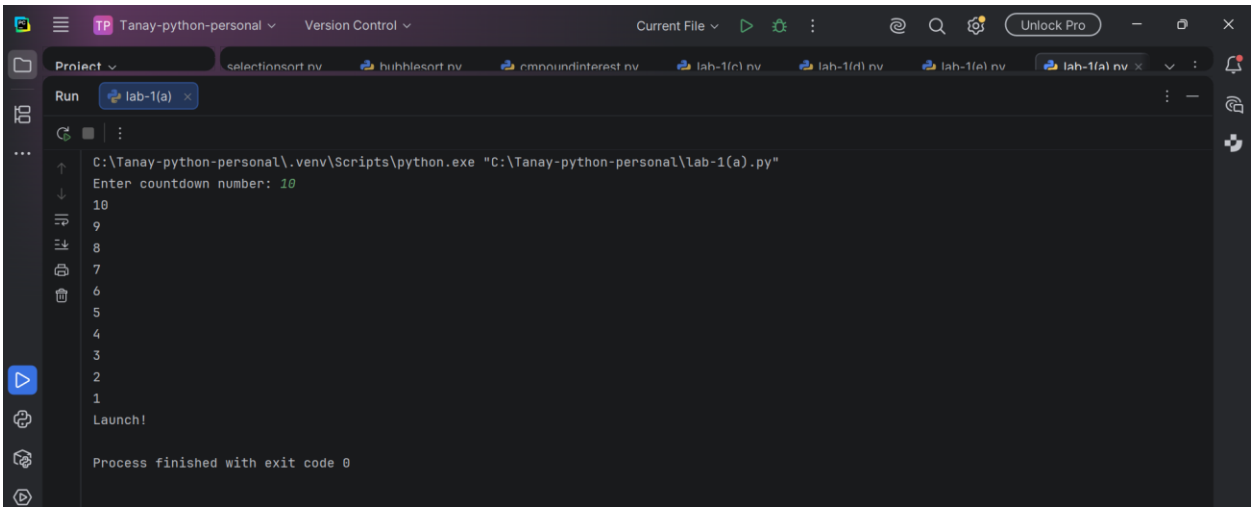
Week1

(a). A rocket launch system displays a countdown before launch. Instead of loop implement countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by “Launch!”.

Code:



Output:

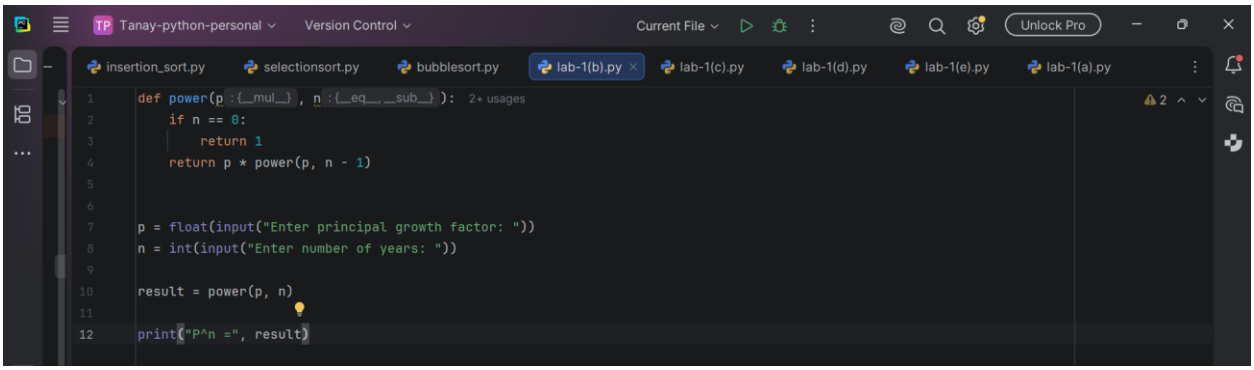


Description:

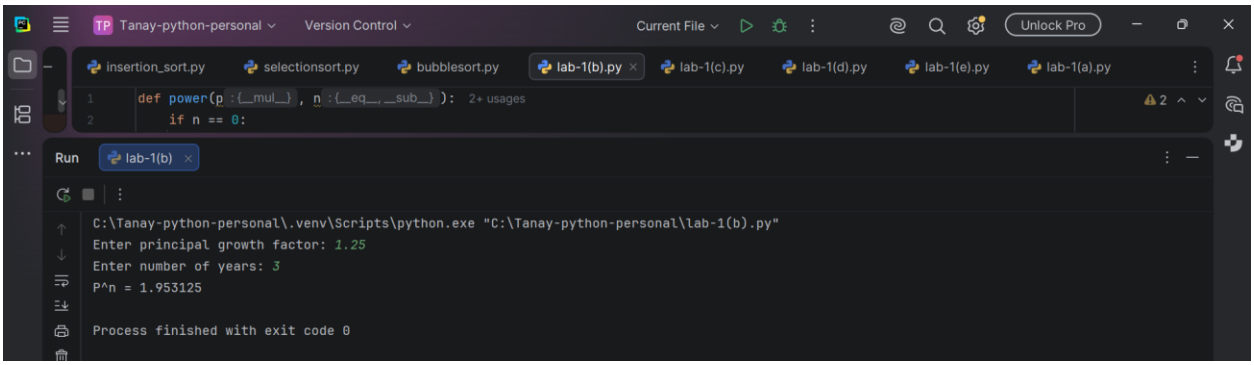
This program displays a rocket launch countdown from n to 1 using recursion instead of loops. The function prints the current countdown number and recursively calls itself with the value decreased by 1. When the value reaches 0, the message “Launch!” is displayed, terminating the recursion.

(b) Banks calculate the growth of an investment using compound interest. The formulae require calculating powers, which can be implemented recursively. Write a recursive program to calculate P^N where:
P=Principal Growth, N=number of years.

Code:



Output:

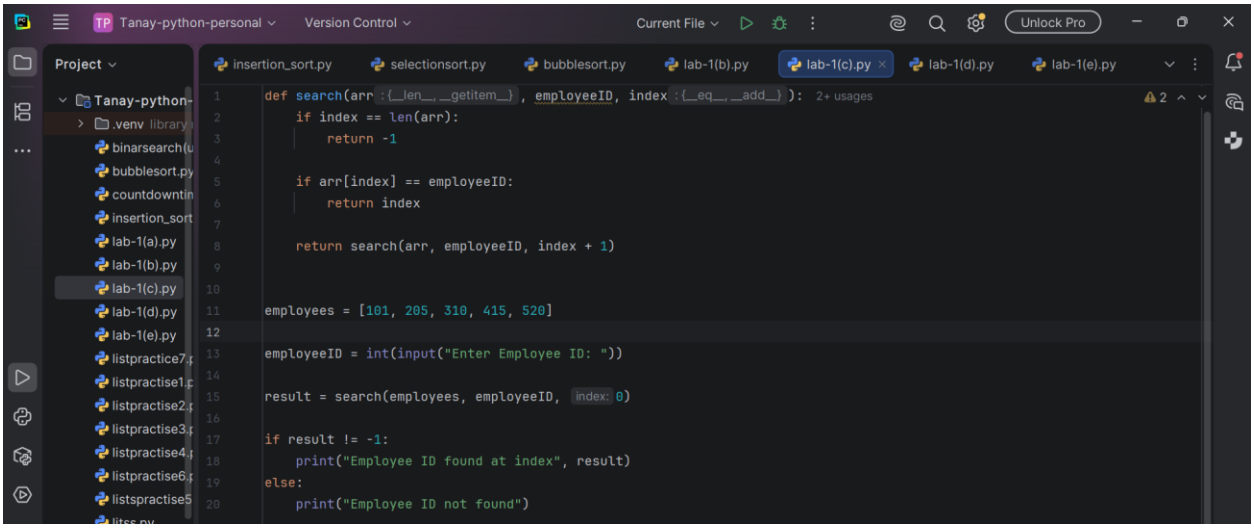


Description:

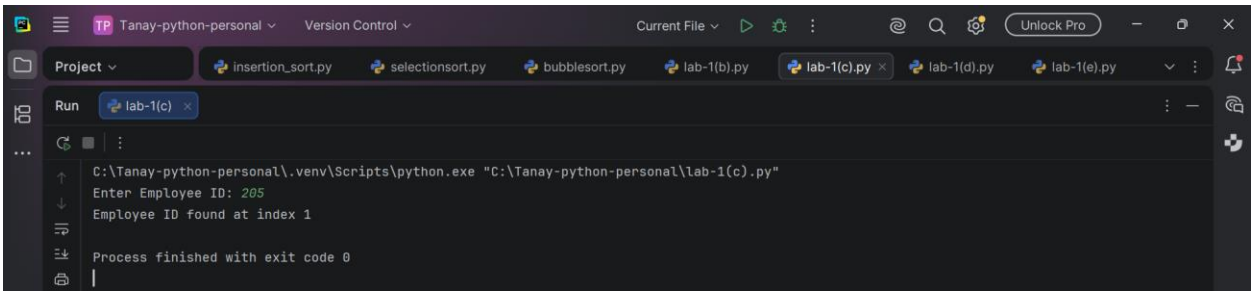
This program calculates P^N using recursion. The function repeatedly multiplies the principal growth factor P by itself while decreasing the value of n by 1. The recursion stops when n becomes 0, since any number raised to power 0 is 1. The final result represents the growth factor of the investment over n years.

(c) An HR department stores ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in the list.

Code:



Output:

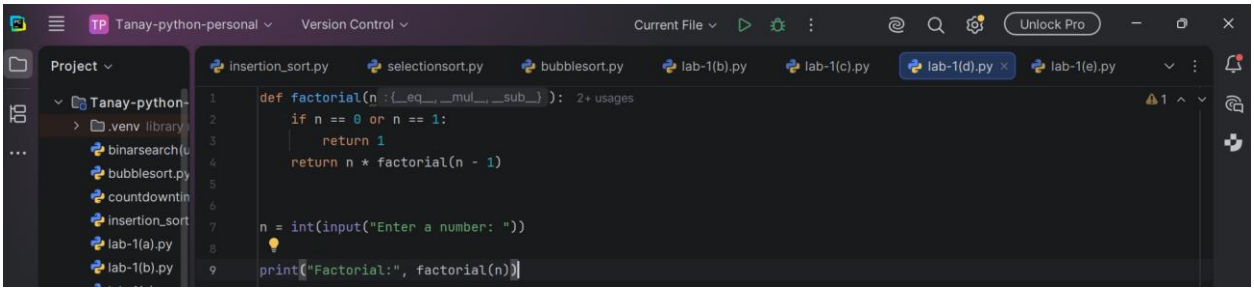


Description:

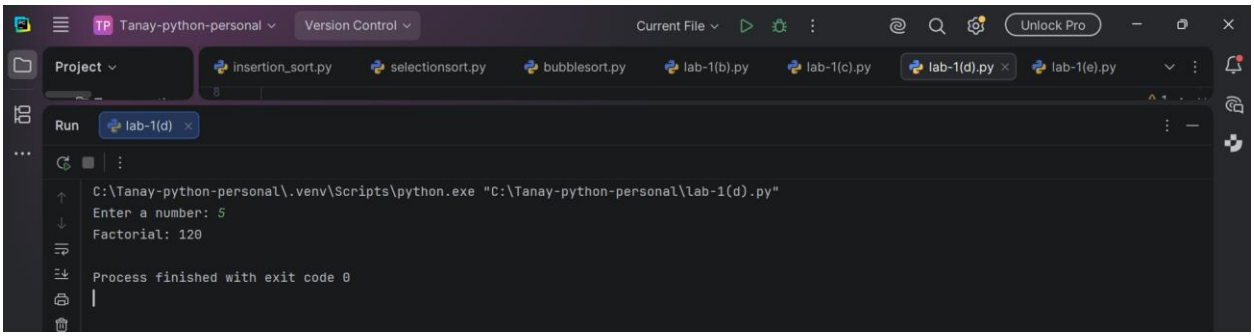
This program searches for a given employee ID in a list using recursion. The function checks each element of the list one by one and recursively moves to the next index until the employee ID is found, or the end of the list is reached. If the ID is found, its index is returned; otherwise, -1 is returned.

(d) In logistics company, develop a recursive program to calculate the factorial of a given number.

Code:



Output:



Description:

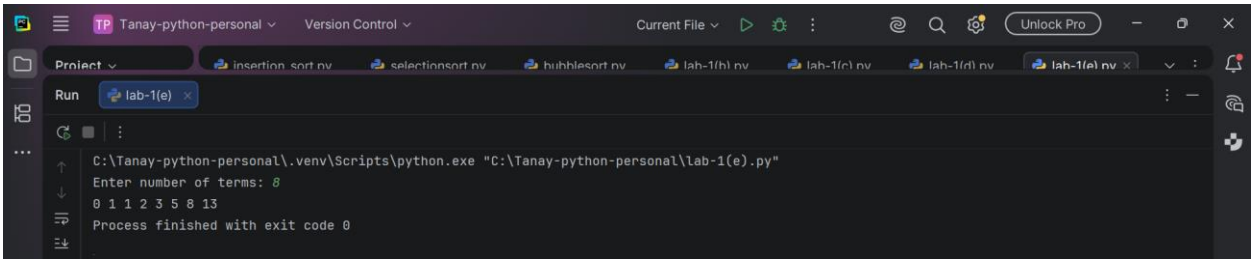
This program calculates the factorial of a given number using recursion. The function multiplies the current number by the factorial of the previous number. The recursion stops when the number becomes 0 or 1, as the factorial of both 0 and 1 is 1.

(e) Write a recursive program to print the first n terms of the Fibonacci series

Code:



Output:



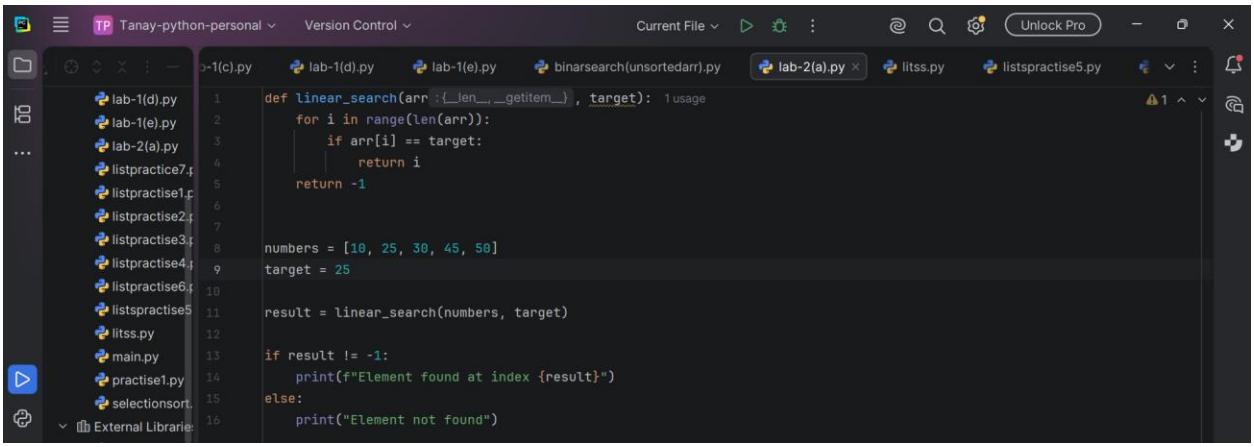
Description:

This program generates the first n terms of the Fibonacci series using recursion. Each Fibonacci term is calculated as the sum of the previous two terms. The base cases are F(0)=0 and F(1)=1. The program recursively calculates and displays each term.

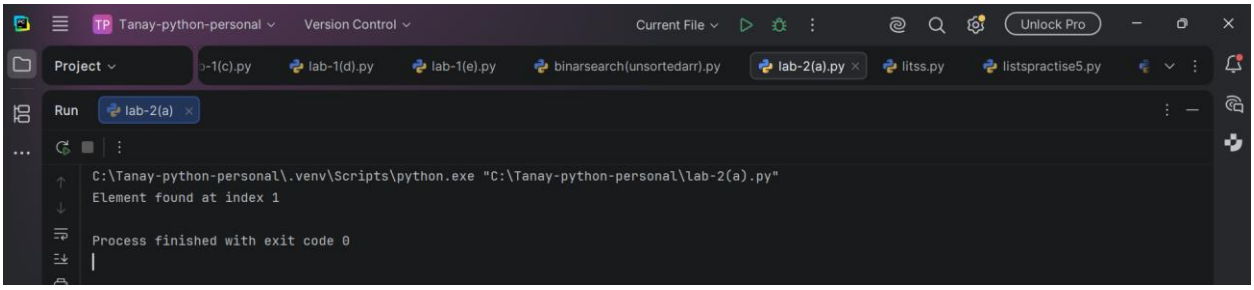
Week-2

(a). Linear Search

Code:



Output:

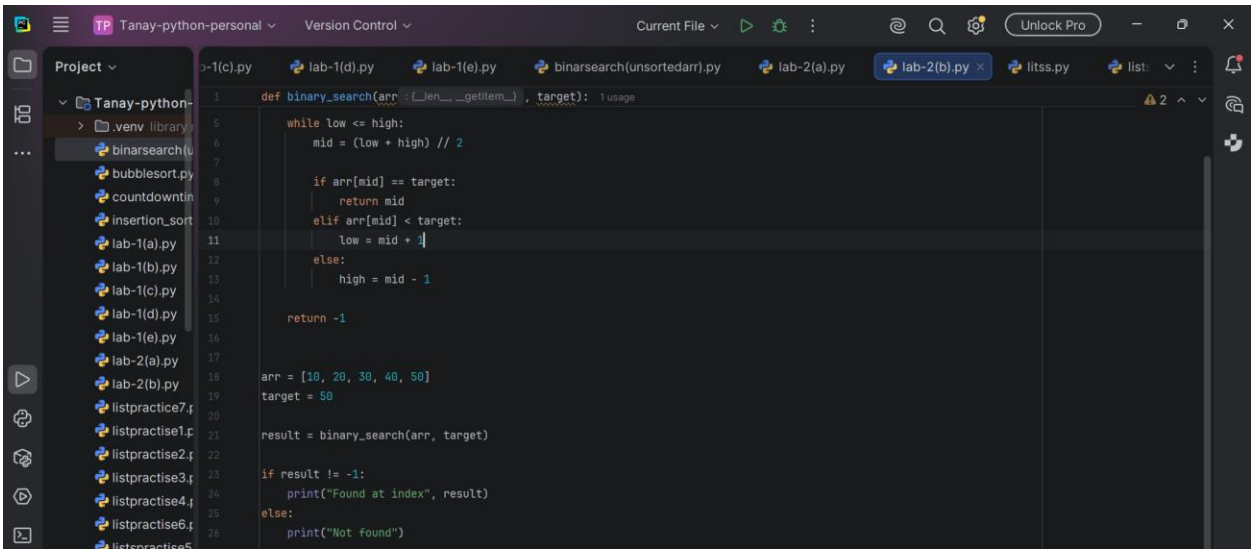


Description:

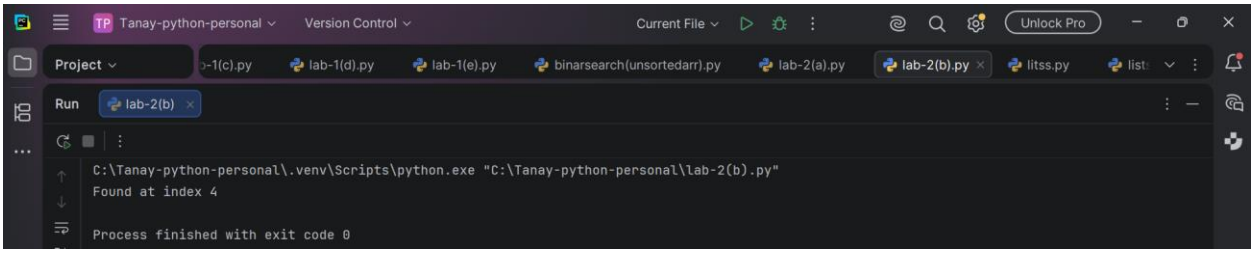
Linear Search is a searching algorithm that checks each element of a list sequentially from the beginning until the required element is found, or the end of the list is reached. It can be used on both sorted and unsorted lists. The time complexity of Linear Search is O(n).

(b). Binary Search

Code:



Output:

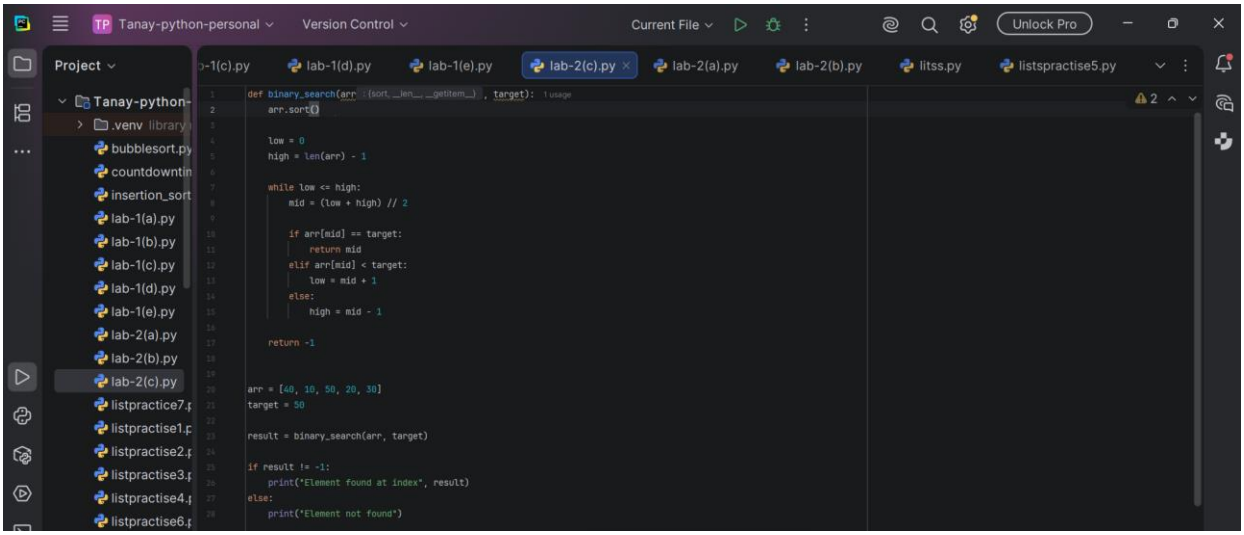


Description:

Binary Search is an efficient searching algorithm used on a sorted list. It compares the target element with the middle element of the list and eliminates half of the search space after each comparison. This process is repeated until the element is found or the search range becomes empty. The time complexity of Binary Search is O(log n).

(c). Write a Python Program to implement Binary Search(unsorted array)

Code:



Output:



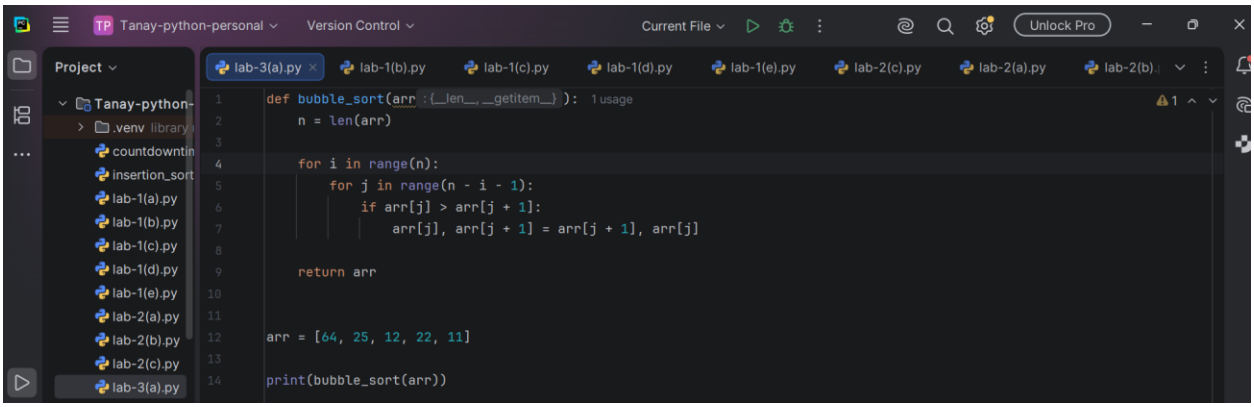
Description:

Binary Search cannot be directly applied to an unsorted list because it requires the elements to be arranged in sorted order. Therefore, the elements must first be arranged in ascending order before applying Binary Search. After sorting, the middle element is compared with the target, and half of the search space is eliminated at each step. The overall time complexity, including sorting, is O (n log n).

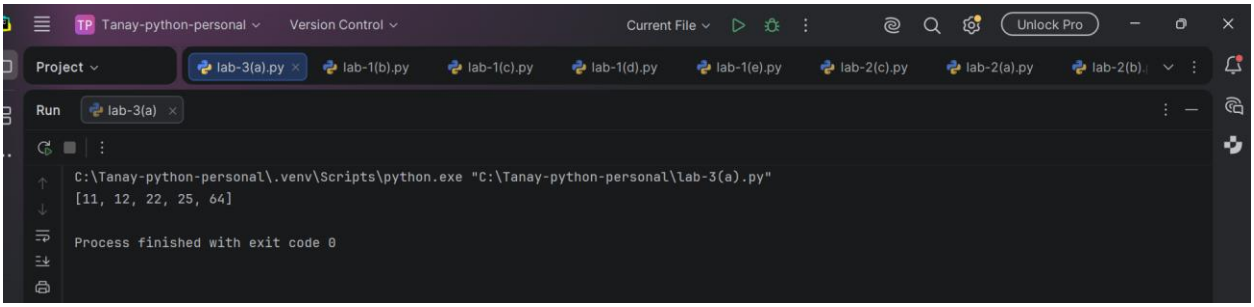
Week-3

(a). Write a Python Program to implement Bubble Sort

Code:



Output:

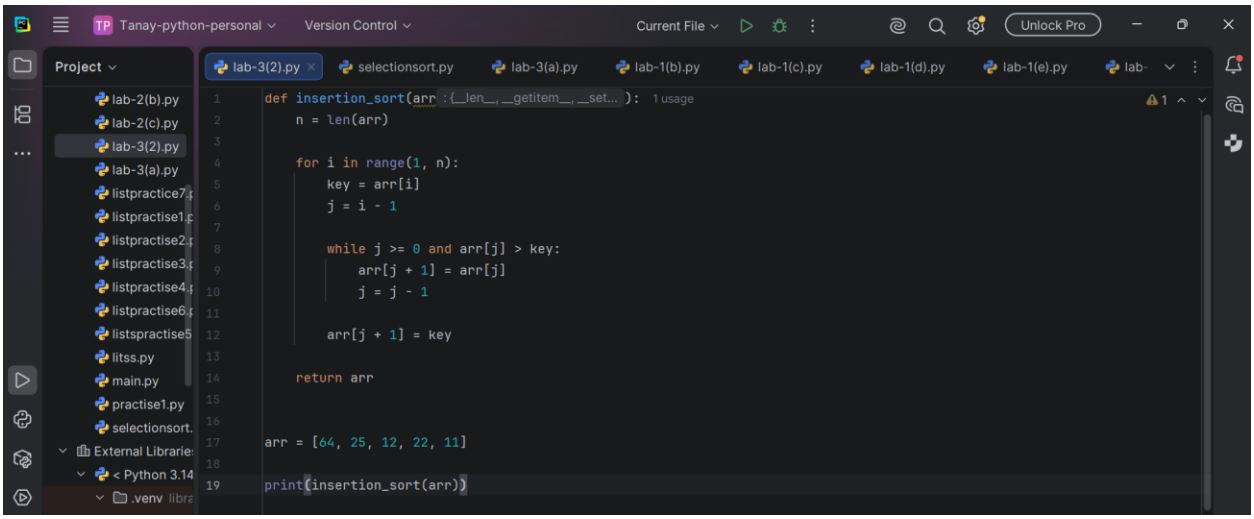


Description:

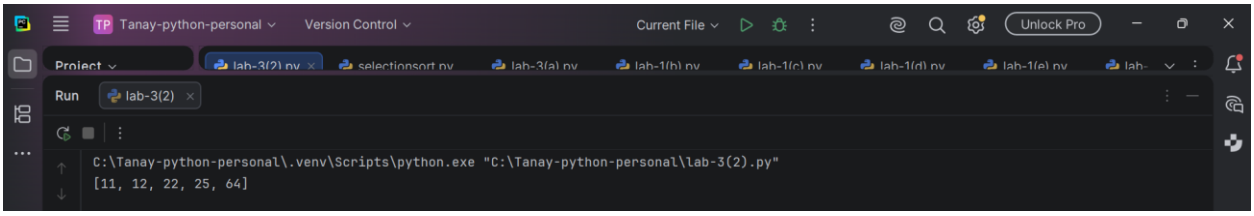
Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. During each pass, the largest unsorted element moves to the end of the list. The process is repeated until the complete list is sorted. The time complexity of Bubble Sort is O (n^2).

(b). Write a Python Program to implement Insertion Sort

Code:



Output:

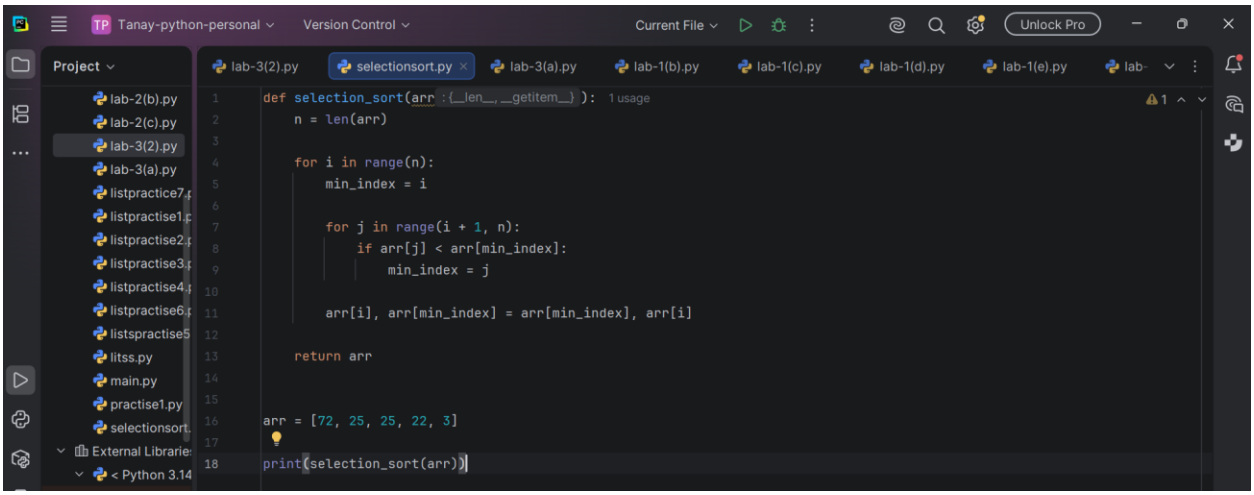


Description:

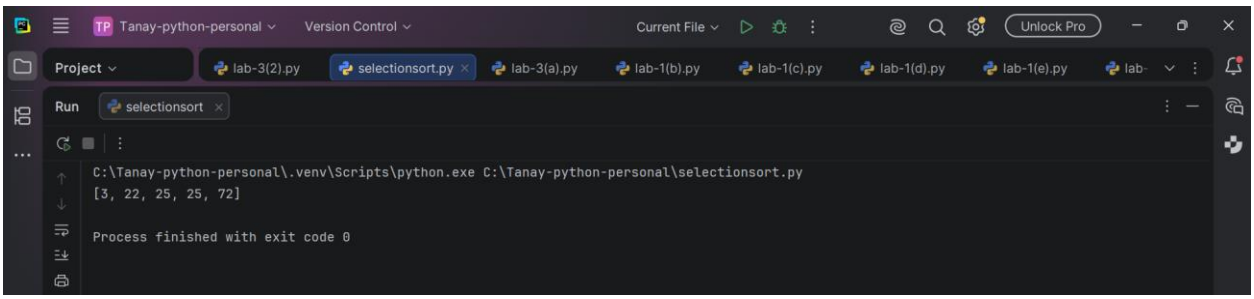
Insertion Sort is a sorting algorithm that builds the sorted list one element at a time. It takes each element from the unsorted portion and inserts it into its correct position in the sorted portion by shifting larger elements to the right. The process continues until all elements are sorted. The worst-case time complexity of Insertion Sort is O(n^2).

(c) Write a Python Program to implement Selection Sort

Code:



Output:



Description:

Selection Sort repeatedly finds the smallest element from the unsorted portion of the list and places it at the beginning of that portion. This process is repeated until all elements are arranged in ascending order. The time complexity of Selection Sort is O(n^2).

Week-4:

4(a) Write a Python Program to implement Merge Sort

Input:

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    while i < len(left):
        result.append(left[i])
        i += 1

    while j < len(right):
        result.append(right[j])
        j += 1

    return result


n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    arr.append(int(input("Enter element: ")))

arr = merge_sort(arr)

print("Sorted array:", arr)
```

Output :

```
C:\Tanay-python-personal\.venv\Scripts\python.exe "C:\DSA_25150\Week04\4(a).py"
Enter number of elements: 5
Enter element: 12
Enter element: 1
Enter element: 34
Enter element: 22
Enter element: 12
Sorted array: [1, 12, 12, 22, 34]

Process finished with exit code 0
```

(b) Write a Python Program to implement Merge Sort

Input:

Tanay-2007-tech Renamed files

Code Blame 28 lines (18 loc) · 487 Bytes Raw

```python
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    left = []
    right = []

    for i in range(len(arr) - 1):
        if arr[i] < pivot:
            left.append(arr[i])
        else:
            right.append(arr[i])

    return quick_sort(left) + [pivot] + quick_sort(right)


n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input("Enter element: ")))

arr = quick_sort(arr)

print("Sorted array:", arr)
```

Output:

```
C:\Tanay-python-personal\.venv\Scripts\python.exe "C:\DSA_25150\Week04\4(b).py"
Enter number of elements: 5
Enter element: 12
Enter element: 21
Enter element: 3
Enter element: 19
Enter element: 45
Sorted array: [3, 12, 19, 21, 45]

Process finished with exit code 0
```

Week-5

(a). Write a Python program to implement SLL and its basic operations

Input:

Tanay-2007-tech Renamed files

Code Blame 130 lines (126 loc) · 3.41 KB Raw

```python
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None
class SinglyLinkedList:
    def __init__(self):
        self.head=None
    def create(self):
      n=int(input("Enter the no of values"))
      for i in range(n):
            x=int(input("Enter the value"))
            new=Node(x)
            if self.head is None:
                self.head=new
            else:
                temp=self.head
                while temp.next:
                    temp=temp.next
                temp.next=new

    def display(self):
     current=self.head
     while current is not None:
        print(current.data,end="->")
        current=current.next
     print("None")
    def insert_begin(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            temp=new
            temp.next=self.head
            self.head=temp
    def insert_end(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            temp=self.head
            while temp.next!=None:
                temp=temp.next
            temp.next=new
    def insert_atindex(self,index,data):
        new = Node(data)
        if index==0:
            new.insert_begin()
        elif self.head is None:
            self.head=new
        else:
            temp = self.head
            for i in range(index-1):
                temp=temp.next
            new.next=temp.next
            temp.next=new
    def deletion_begin(self):
        if self.head is None:
            print("List is empty")
        else:
           self.head=self.head.next
    def deletion_end(self):
```

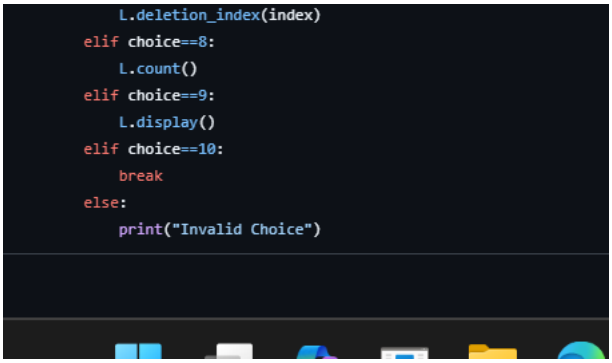
```
    def deletion_end(self):
        if self.head is None:
            print("List is empty")
        else:
            temp=self.head
            while temp.next.next!=None:
                temp=temp.next
            temp.next=None
    def deletion_index(self,index):
        if index<0:
            print("Invalid index")
        elif self.head is None:
            print("Deletion cant be performed")
        elif index==0:
            self.deletion_begin()
        else:
            temp = self.head
            for i in range(index-1):
                temp=temp.next
            temp.next=temp.next.next
    def count(self):
        c=0
        temp=self.head
        while temp!=None:
            c+=1
            temp=temp.next
        print("The number of nodes: ",c)

L=SinglyLinkedList()
while True:
    print("1.Create")
    print("2.Insertion at begin")
```

```
    def count(self):
    print("2.Insertion at begin")
    print("3.Insertion at end")
    print("4.Insertion at specific Index")
    print("5.Deletion at begin")
    print("6.Deletion at end ")
    print("7.Deletion at specific index")
    print("8.Count Number of nodes: ")
    print("9.Display")
    print("10.Exit")

    choice=int(input("Enter your Choice: "))

    if choice==1:
        L.create()
    elif choice==2:
        x = int(input("Enter value: "))
        L.insert_begin(x)
    elif choice==3:
        x = int(input("Enter value: "))
        L.insert_end(x)
    elif choice==4:
        index=int(input("Enter index"))
        x = int(input("Enter value: "))
        L.insert_atindex(index,x)
    elif choice==5:
        L.deletion_begin()
    elif choice==6:
        L.deletion_end()
    elif choice==7:
        index = int(input("Enter value: "))
        L.deletion_index(index)
```



```
            L.deletion_index(index)
        elif choice==8:
            L.count()
        elif choice==9:
            L.display()
        elif choice==10:
            break
        else:
            print("Invalid Choice")
```

Output:

```
C:\Tanay-python-personal\.venv\Scripts\python.exe "C:\DSA_25150\Week05\5(a).py"
1.Create
2.Insertion at begin
3.Insertion at end
4.Insertion at specific Index
5.Deletion at begin
6.Deletion at end
7.Deletion at specific index
8.Count Number of nodes:
9.Display
10.Exit
Enter your Choice: 1
Enter the no of values6
Enter the value10
Enter the value20
Enter the value30
Enter the value40
Enter the value50
Enter the value60
1.Create
2.Insertion at begin
3.Insertion at end
4.Insertion at specific Index
5.Deletion at begin
6.Deletion at end
7.Deletion at specific index
8.Count Number of nodes:
9.Display
10.Exit
Enter your Choice: 9
10->20->30->40->50->60->None
1.Create
2.Insertion at begin
3.Insertion at end
4.Insertion at specific Index
5.Deletion at begin
6.Deletion at end
7.Deletion at specific index
8.Count Number of nodes:
9.Display
10.Exit
Enter your Choice: 10

Process finished with exit code 0
```

(b) Write a Python program to implement DLL with the basic operations

Input:

Tanay-2007-tech Renamed files

Code Blame 192 lines (148 loc) · 4.29 KB Raw

```python

class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter the no of elements: "))
        for i in range(n):
            x = int(input("Enter the value: "))
            new = Node(x)

            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new
                new.prev = temp

    def display(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head
        while current is not None:
            print(current.data, end="<->")
            current = current.next
        print("None")

    def insert_begindl(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            new.next = self.head
            self.head.prev = new
            self.head = new

    def insert_enddl(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new
            new.prev = temp

    def insert_at_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_begindl(data)
            return

        if self.head is None:
            print("Invalid index")
            return

        new = Node(data)
        temp = self.head

        for i in range(index - 1):
            temp = temp.next
            if temp is None:
                print("Invalid index")
                return

        new.next = temp.next
        new.prev = temp

        if temp.next:
```

```
            temp.next = new

    def delete_atbegindl(self):
        if self.head is None:
            print("Delete operation can't be performed")
            return

        if self.head.next is None:
            self.head = None
        else:
            self.head.next.prev = None
            self.head = self.head.next

    def delete_atenddl(self):
        if self.head is None:
            print("Delete operation can't be performed")
            return

        if self.head.next is None:
            self.head = None
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.prev.next = None

    def delete_at_index(self, index):
        if index < 0:
            print("Invalid index")
            return

        if self.head is None:
            print("Deletion operation can't be performed")
            return

        if index == 0:
            self.delete_atbegindl()
            return

        temp = self.head

        for i in range(index):
```

```

        for i in range(index):
            temp = temp.next
            if temp is None:
                print("Invalid index")
                return

        if temp.next is None:
            temp.prev.next = None
        else:
            temp.prev.next = temp.next
            temp.next.prev = temp.prev


dl = DoublyLinkedList()

while True:
    print("\n1. Create")
    print("2. Display")
    print("3. Insert at Beginning")
    print("4. Insert at End")
    print("5. Insert at Index")
    print("6. Delete at Beginning")
    print("7. Delete at End")
    print("8. Delete at Index")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dl.create()

    elif choice == 2:
        dl.display()

    elif choice == 3:
        data = int(input("Enter the value: "))
        dl.insert_begindl(data)

    elif choice == 4:
        data = int(input("Enter the value: "))
        dl.insert_enddl(data)
```

```python

    elif choice == 5:
        index = int(input("Enter the index: "))
        data = int(input("Enter the value: "))
        dl.insert_at_index(index, data)

    elif choice == 6:
        dl.delete_atbegindl()

    elif choice == 7:
        dl.delete_atenddl()

    elif choice == 8:
        index = int(input("Enter the index: "))
        dl.delete_at_index(index)

    elif choice == 9:
        break

    else:
        print("Invalid choice")
```

Output

```
C:\Tanay-python-personal\.venv\Scripts\python.exe "C:\DSA_25150\Week05\5(b).py"

1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 1
Enter the no of elements: 5
Enter the value: 10
Enter the value: 30
Enter the value: 50
Enter the value: 70
Enter the value: 90
```

```
1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 2
10<->30<->50<->70<->90<->None
```

```
1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 9

Process finished with exit code 0
```

(c) Write a Python program to implement CLL with the basic operations

Code Blame 213 lines (154 loc) · 4.61 KB

```python

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter the no of elements: "))

        for i in range(n):
            x = int(input("Enter the value: "))
            new = Node(x)

            if self.head is None:
                self.head = new
                new.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                temp.next = new
                new.next = self.head

    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while True:
            print(temp.data, end="->")
            temp = temp.next

            if temp == self.head:
                break

        print("(HEAD)")
```

```python
class CircularLinkedList:
    def display(self):

    def insert_begincll(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new.next = self.head
            temp.next = new
            self.head = new

    def insert_endcll(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new
            new.next = self.head

    def insert_at_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_begincll(data)
            return

        if self.head is None:
            print("Invalid index")
            return
```

```
        temp = self.head

        for i in range(index - 1):
            temp = temp.next

            if temp == self.head:
                print("Invalid index")
                return

        new = Node(data)
        new.next = temp.next
        temp.next = new

    def delete_at_begincll(self):
        if self.head is None:
            print("Delete operation can't be performed")
            return

        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = self.head.next
            self.head = self.head.next

    def delete_at_endcll(self):
        if self.head is None:
            print("Delete operation can't be performed")
            return

        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head

            while temp.next.next != self.head:
                temp = temp.next

            temp.next = self.head
```

```

    def delete_at_index(self, index):
        if index < 0:
            print("Invalid index")
            return

        if self.head is None:
            print("Deletion operation can't be performed")
            return

        if index == 0:
            self.delete_at_begincll()
            return

        temp = self.head

        for i in range(index - 1):
            temp = temp.next

            if temp == self.head:
                print("Invalid index")
                return

        if temp.next == self.head:
            print("Invalid index")
            return

        temp.next = temp.next.next


cll = CircularLinkedList()

while True:
    print("\n1. Create")
    print("2. Display")
    print("3. Insert at Beginning")
    print("4. Insert at End")
    print("5. Insert at Index")
    print("6. Delete at Beginning")
    print("7. Delete at End")
    print("8. Delete at Index")
    print("9. Exit")

    choice = int(input("Enter your choice: "))
```

```python
    if choice == 1:
        cll.create()

    elif choice == 2:
        cll.display()

    elif choice == 3:
        data = int(input("Enter the value: "))
        cll.insert_begincll(data)

    elif choice == 4:
        data = int(input("Enter the value: "))
        cll.insert_endcll(data)

    elif choice == 5:
        index = int(input("Enter the index: "))
        data = int(input("Enter the value: "))
        cll.insert_at_index(index, data)

    elif choice == 6:
        cll.delete_at_begincll()

    elif choice == 7:
        cll.delete_at_endcll()

    elif choice == 8:
        index = int(input("Enter the index: "))
        cll.delete_at_index(index)

    elif choice == 9:
        break

    else:
        print("Invalid choice")
```

Output:

```
C:\Tanay-python-personal\.venv\Scripts\python.exe "C:\DSA_25150\Week05\5(c).py"

1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 1
Enter the no of elements: 5
Enter the value: 20
Enter the value: 40
Enter the value: 60
Enter the value: 80
Enter the value: 100
```

```
1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 2
20->40->60->80->100->(HEAD)
```

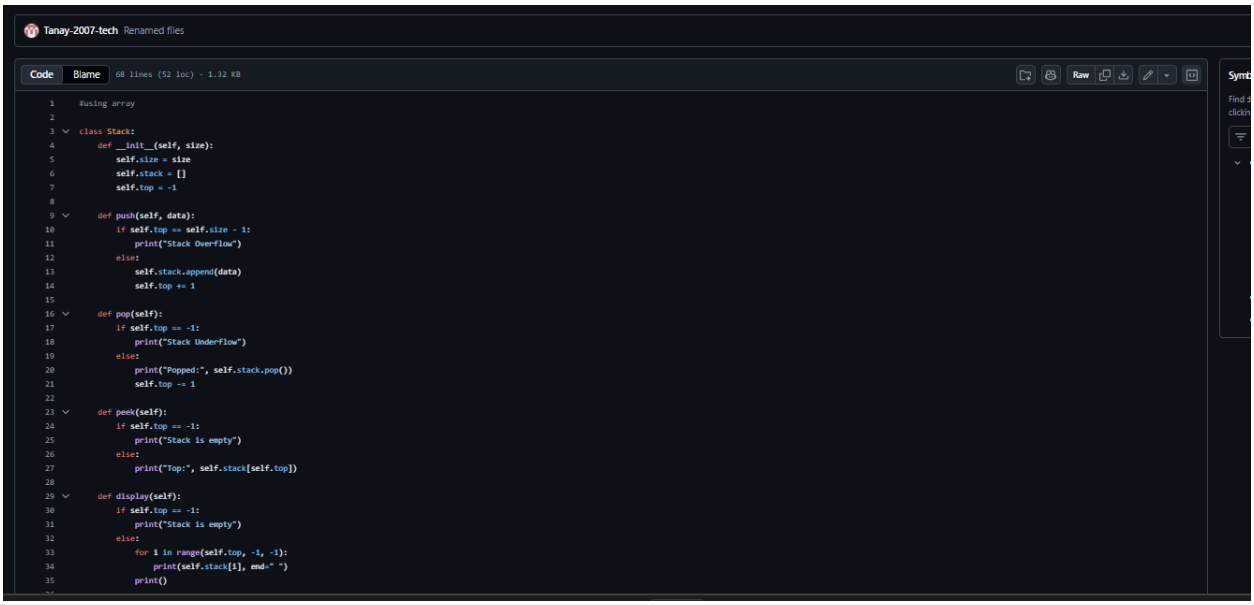
```
1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 9

Process finished with exit code 0
```

Week-6

(a) Write a Python program to implement Stack operations such as Push, Pop, Peek, and Display using an array.

Input:




```python
#using array

class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = []
        self.top = -1

    def push(self, data):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.stack.append(data)
            self.top += 1

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            print("Popped:", self.stack.pop())
            self.top -= 1

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            for i in range(self.top, -1, -1):
                print(self.stack[i], end=" ")
            print()
```



```python

size = int(input("Enter stack size: "))
s = Stack(size)

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the value: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

Output:

```
C:\Tanay-python-personal\.venv\Scripts\python.exe "C:\DSA_25150\Week06\6(a).py"
Enter stack size: 5

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 12

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 13

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 14

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 15

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
15 14 13 12

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5

Process finished with exit code 0
```

(b). Write a Python program to implement Stack operations such as Push, Pop, Peek using linked list.

Input:

Tanay-2007-tech Renamed files

Code Blame 71 lines (52 loc) · 1.29 KB Raw

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped:", self.top.data)
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top

            while temp is not None:
                print(temp.data, end=" ")
                temp = temp.next

            print()


s = Stack()

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the value: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

Output:

```
C:\Tanay-python-personal\.venv\Scripts\python.exe "C:\DSA_25150\Week06\6(b).py"

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 10

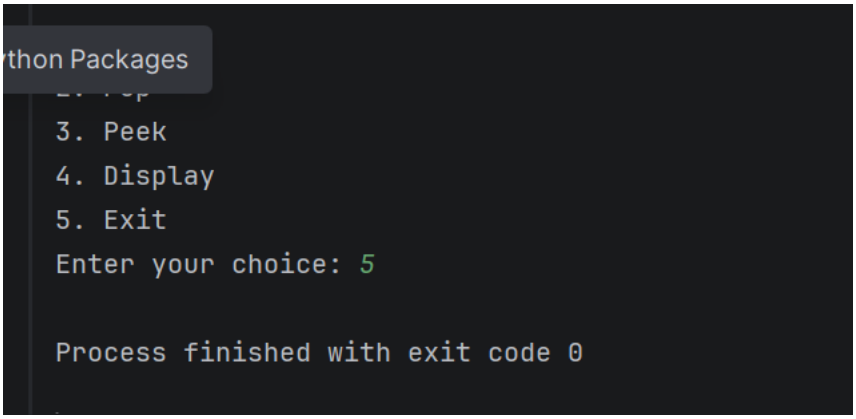
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 20
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 30

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 40
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
40 30 20 10

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top: 40
```



Description:

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle, in which the element inserted last is removed first. This program implements stack operations using a singly linked list in Python with a menu-driven approach.

Each node in the linked list contains data and a pointer to the next node. A top pointer is used to keep track of the topmost element of the stack.

The program supports the following operations:

1. **Push:** Inserts a new element at the top of the stack by creating a new node and updating the top pointer.
2. **Pop:** Removes the top element from the stack and updates the top pointer to the next node. If the stack is empty, an appropriate message is displayed.
3. **Peek:** Displays the top element without removing it from the stack. If the stack is empty, an appropriate message is displayed.

The linked list implementation allows the stack to grow dynamically as new nodes are added, without requiring a fixed-size array. The program uses a menu-driven approach to allow the user to perform stack operations repeatedly until the exit option is selected.

Week-7

(a) Write a Python program to implement a Queue and its **o**perations such as Enqueue, Dequeue, Peek, and Display with a menu-driven approach.

Input:

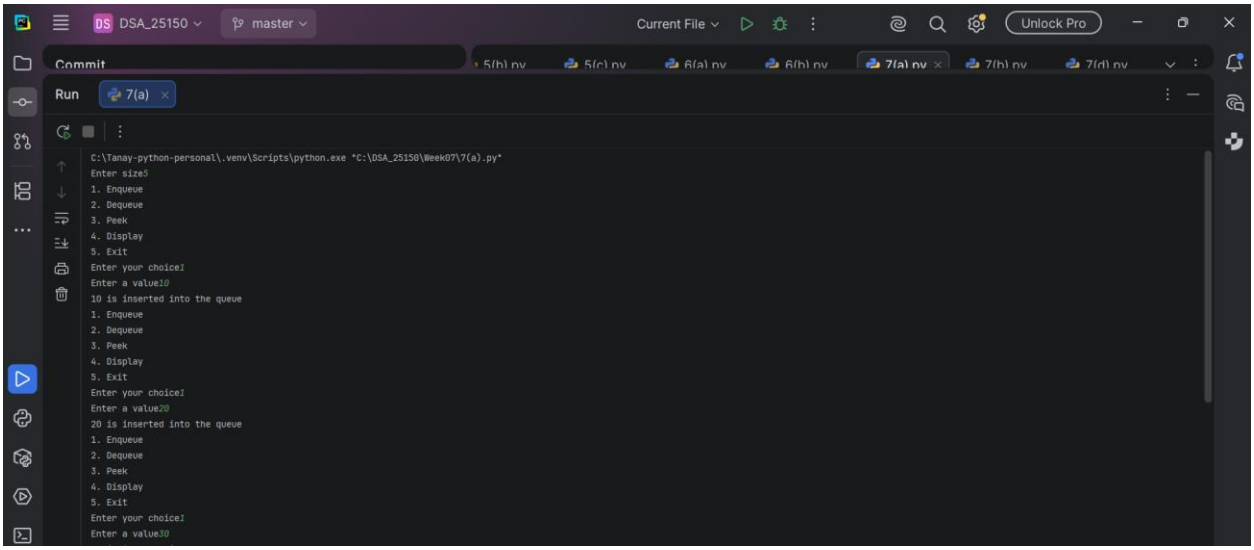
Tanay-2007-tech WEEK 7

Code Blame 59 lines (59 loc) · 1.63 KB Raw

```python
class QueueArray:
    def __init__(self,size):
        self.size=size
        self.queue=[None]*size
        self.front=-1
        self.rear=-1
    def enqueue(self,data):
        if self.rear==self.size-1:
            print("Queue Overflow")
        else:
            if self.front==-1:
                self.front=0
            self.rear+=1
            self.queue[self.rear]=data
            print(f"{data} is inserted into the queue")
    def dequeue(self):
        if self.front==-1 or self.front>self.rear:
            print("Queue underflow")
        else:
            x=self.queue[self.front]
            self.queue[self.front]=None
            self.front+=1
            print(f"{x} is deleted from the queue")
            if self.front>self.rear:
                self.front=-1
                self.rear=-1
    def peek(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            print(f"Peek: {self.queue[self.front]}")
    def display(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            for i in range(self.front,self.rear+1):
                print(self.queue[i])
size=int(input("Enter size"))
Queue=QueueArray(size)
while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice=int(input("Enter your choice"))
    if choice==1:
        data=int(input("Enter a value"))
        Queue.enqueue(data)
    elif choice==2:
        Queue.dequeue()
```

```
52          elif choice==3:
53              Queue.peek()
54          elif choice==4:
55              Queue.display()
56          elif choice==5:
57              break
58          else:
59              print("Invalid choice")
```

Output:



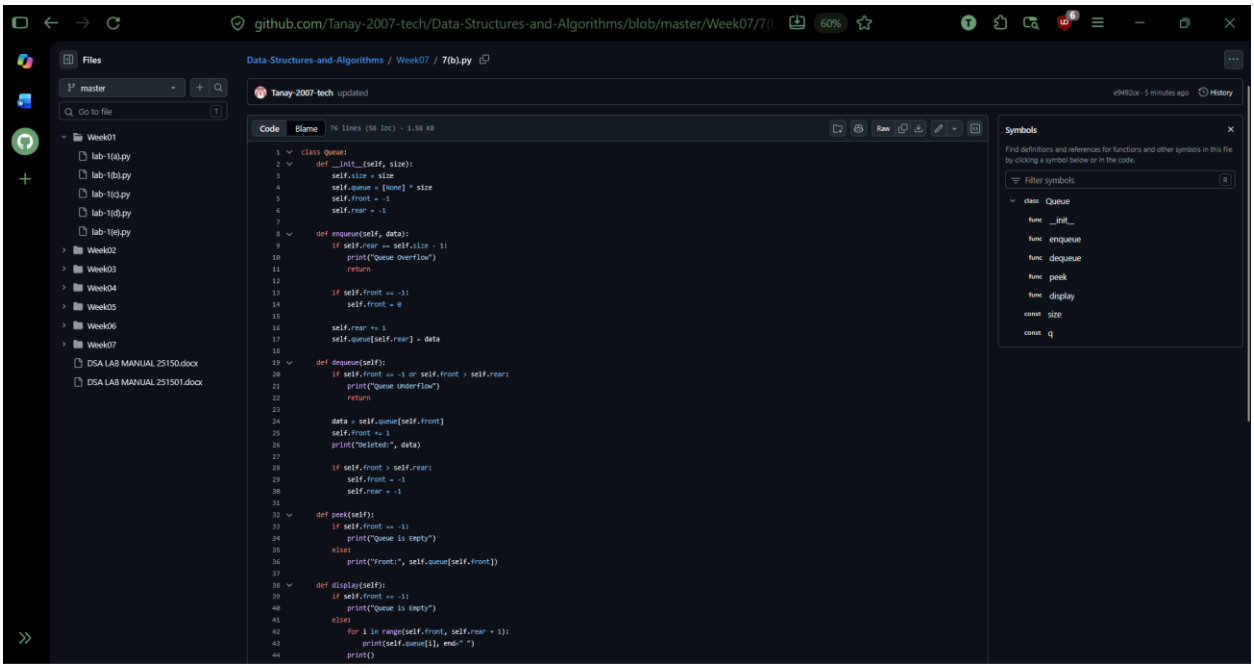


```
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice5

Process finished with exit code 0
```

(b) Implement using array and SLL.

Input: using array



```
size = int(input("Enter size of queue: "))
q = Queue(size)

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter Your Choice: "))

    if choice == 1:
        data = int(input("Enter value: "))
        q.enqueue(data)

    elif choice == 2:
        q.dequeue()

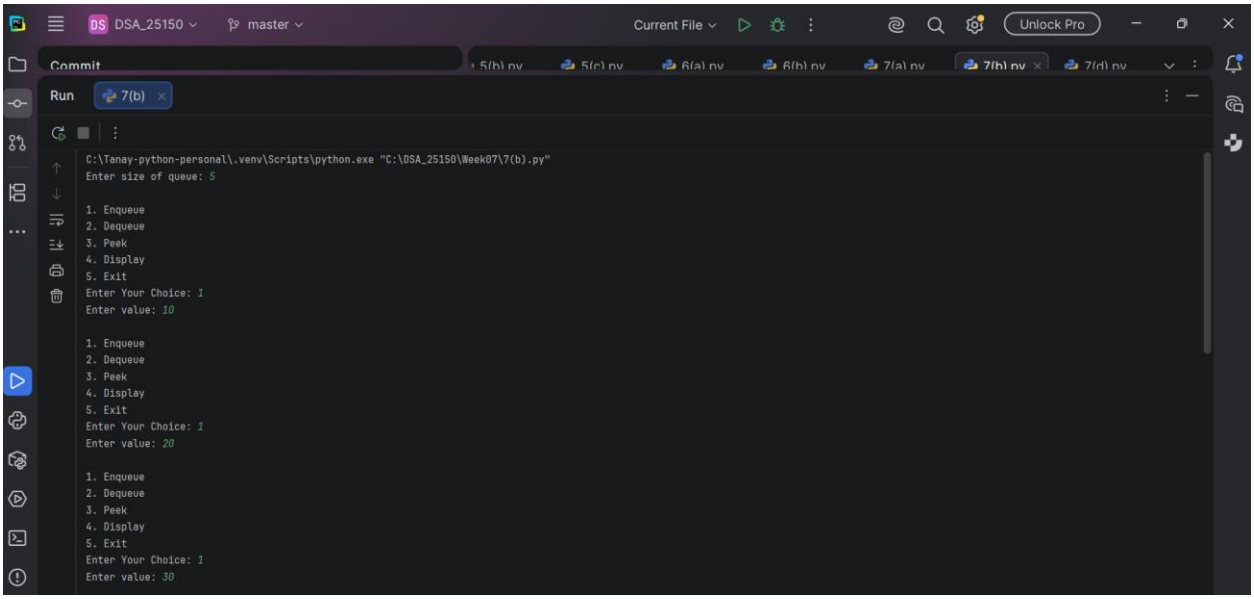
    elif choice == 3:
        q.peek()

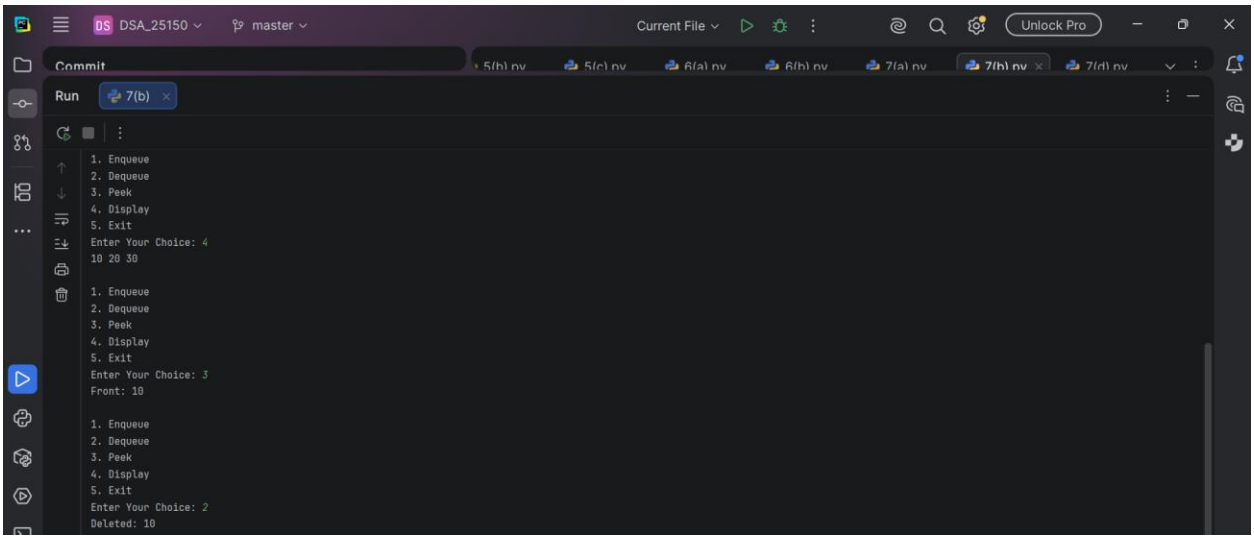
    elif choice == 4:
        q.display()

    elif choice == 5:
        break

    else:
        print("Invalid Choice")
```

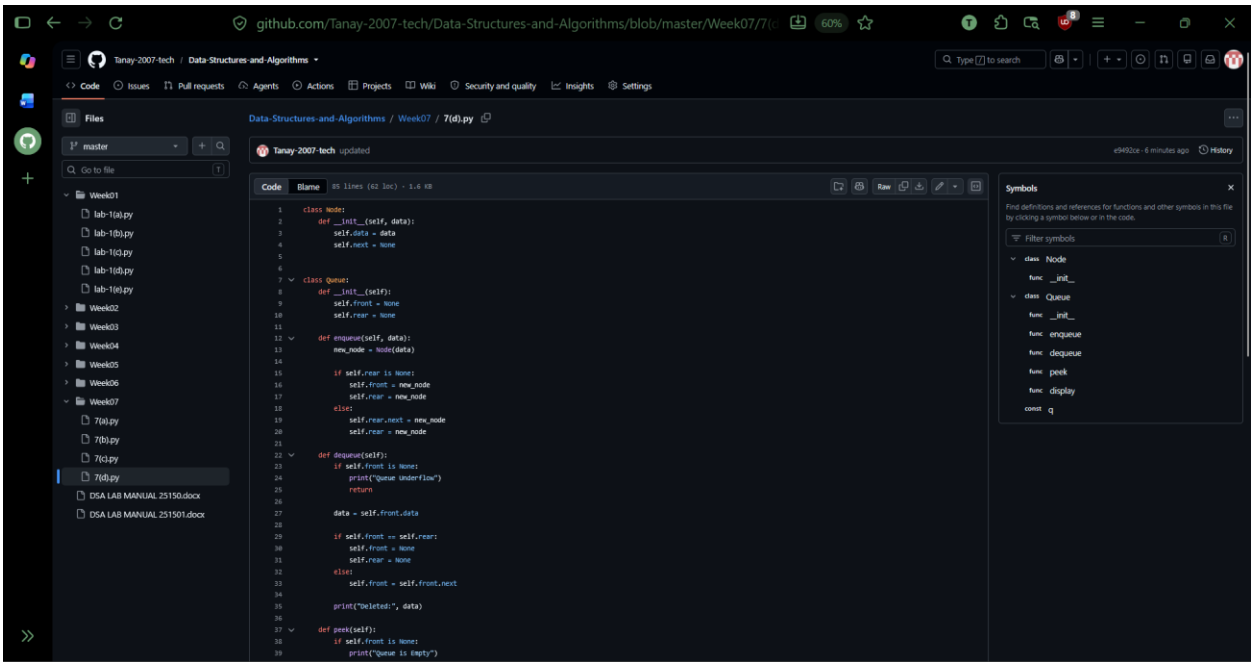
Output:




DSA_25150
master
Current File
Unlock Pro
Commit
5(b).py
5(c).py
6(a).py
6(b).py
7(a).py
7(b).py
7(d).py
Run
7(b)
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Your Choice: 4
10 20 30
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Your Choice: 3
Front: 10
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Your Choice: 2
Deleted: 10


Enter Your Choice: 4
20 30
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Your Choice: 5
Process finished with exit code 0

Using SLL:


github.com/Tanay-2007-tech/Data-Structures-and-Algorithms/blob/master/Week07/7(
Tanay-2007-tech / Data-Structures-and-Algorithms
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
master
Week01
lab-1(a).py
lab-1(b).py
lab-1(c).py
lab-1(d).py
lab-1(e).py
Week02
Week03
Week04
Week05
Week06
Week07
7(a).py
7(b).py
7(c).py
7(d).py
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
Data-Structures-and-Algorithms / Week07 / 7(d).py
Tanay-2007-tech updated
Code
Blame
Raw
Symbols
Node
__init__
Queue
__init__
enqueue
dequeue
peek
display
q

```
            else:
                print("Front:", self.front.data)

        def display(self):
            if self.front is None:
                print("Queue is Empty")
                return

            temp = self.front

            while temp is not None:
                print(temp.data, end=" ")
                temp = temp.next

            print()


    q = Queue()

    while True:
        print("\n1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = int(input("Enter Your Choice: "))

        if choice == 1:
            data = int(input("Enter value: "))
            q.enqueue(data)

        elif choice == 2:
            q.dequeue()

        elif choice == 3:
            q.peek()

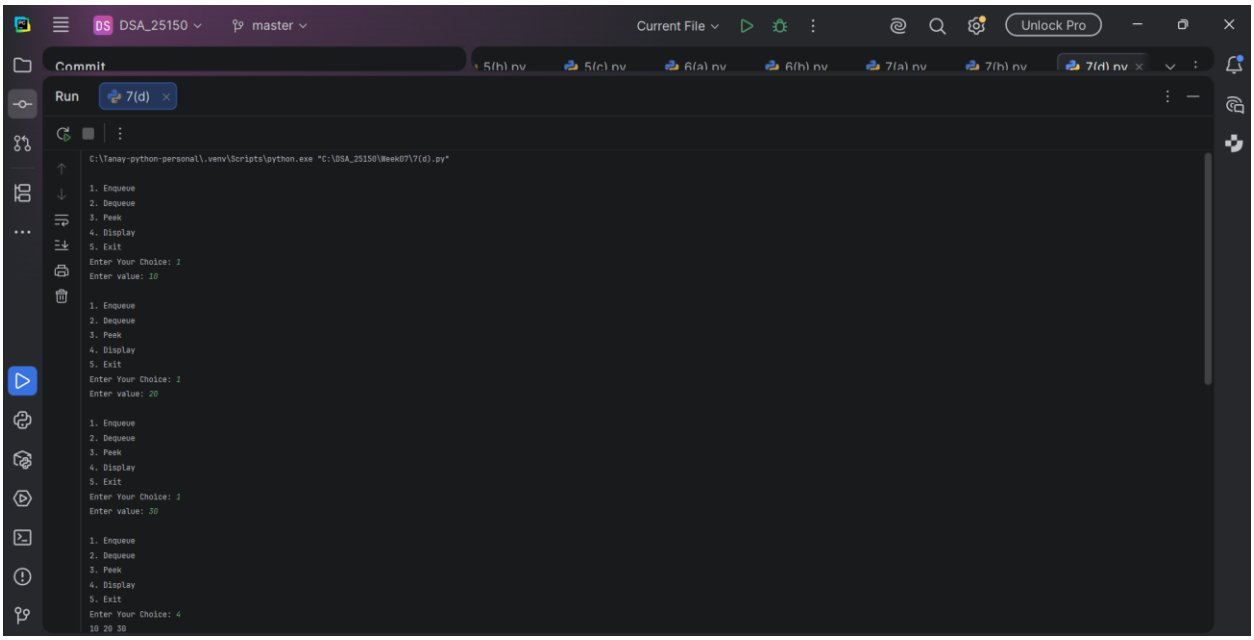
        elif choice == 4:
            q.display()

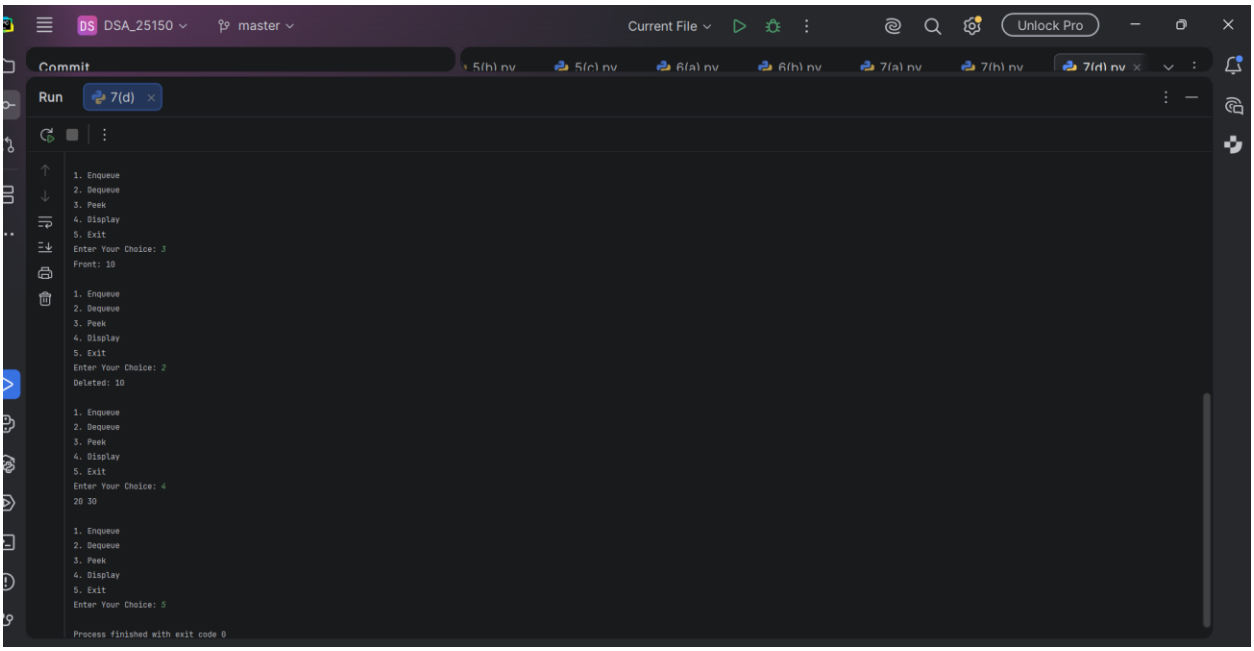
        elif choice == 5:
            break
```

```
        elif choice == 5:
            break

        else:
            print("Invalid Choice")
```

Output:





(c) Write a Python Program to implement Circular Queue

```python
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, data):
        if (self.rear + 1) % self.size == self.front:
            print("Queue is full")
            return

        if self.front == -1:
            self.front = 0
            self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.size

        self.queue[self.rear] = data

    def dequeue(self):
        if self.front == -1:
            print("Queue is empty")
            return

        data = self.queue[self.front]
```

```
            data = self.queue[self.front]

            if self.front == self.rear:
                self.front = -1
                self.rear = -1
            else:
                self.front = (self.front + 1) % self.size

            print("Deleted:", data)

    def peek(self):
            if self.front == -1:
                print("Queue is empty")
            else:
                print("Front:", self.queue[self.front])

    def display(self):
            if self.front == -1:
                print("Queue is empty")
                return

            i = self.front

            while True:
                print(self.queue[i], end=" ")
```

```
    def display(self):
        print("\n1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = int(input("Enter choice: "))

        if choice == 1:
            data = int(input("Enter data: "))
            cq.enqueue(data)

        elif choice == 2:
            cq.dequeue()

        elif choice == 3:
            cq.peek()

        elif choice == 4:
            cq.display()

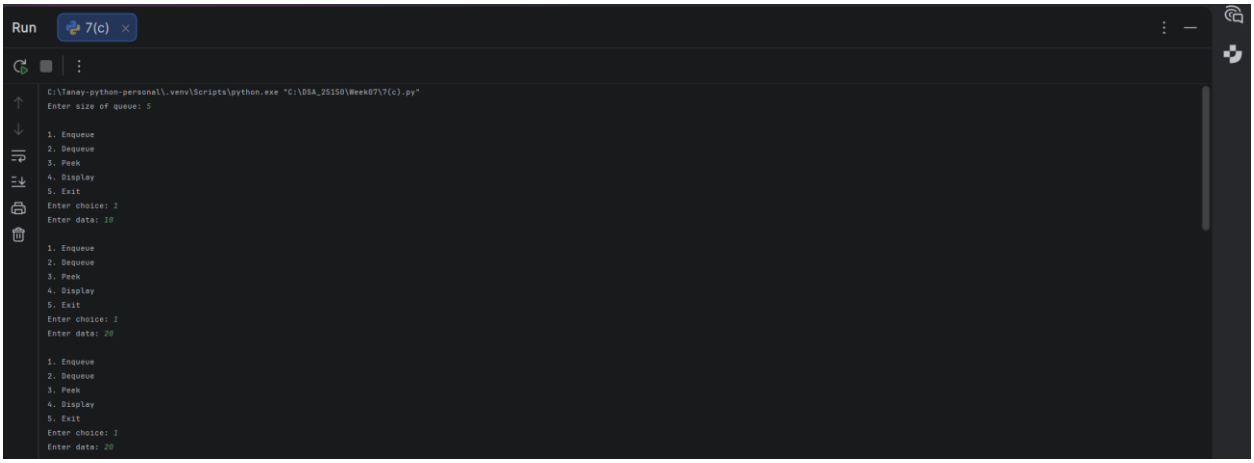
        elif choice == 5:
            break

        else:
```

```
        elif choice == 5:
            break

        else:
            print("Invalid choice")
```

Output:









Description:

A queue is a linear data structure that follows the FIFO (First In, First Out) principle, in which the element inserted first is removed first. This program implements a queue using an array and a singly linked list (SLL) with a menu-driven approach.

The program supports the following operations:

1. **Enqueue:** Inserts a new element at the rear end of the queue.
2. **Dequeue:** Removes an element from the front of the queue. If the queue is empty, an appropriate message is displayed.
3. **Peek:** Displays the front element of the queue without removing it.
4. **Display:** Displays all the elements present in the queue.

In the array implementation, elements are stored in an array, and front and rear indices are used to manage the queue operations. In the singly linked list implementation, each node contains data and a pointer to the next node. The front and rear pointers are used to insert and delete elements efficiently.

The menu-driven approach allows the user to select an operation repeatedly until the exit option is chosen.

A queue is a linear data structure that follows the FIFO (First In, First Out) principle, in which the element inserted first is removed first. A circular queue is an extension of a queue in which the last position is connected to the first position, allowing unused spaces in the array to be reused efficiently.

This program implements queue operations using an array, a circular array, and a singly linked list (SLL) with a menu-driven approach. The following operations are performed:

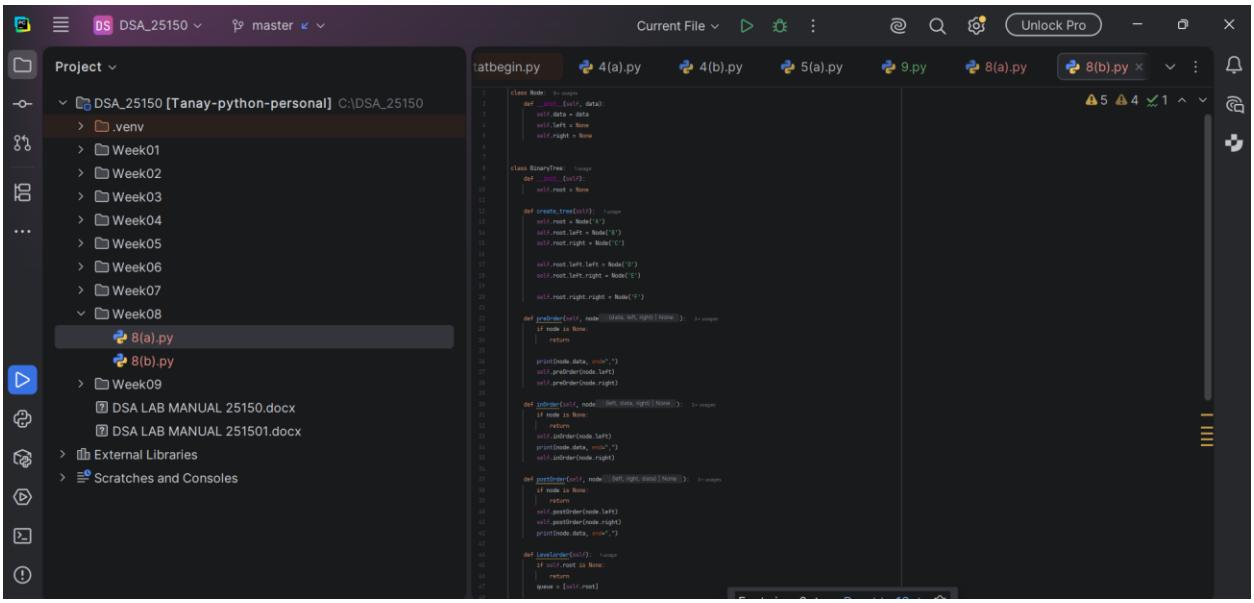
1. **Enqueue:** Inserts a new element at the rear end of the queue.
2. **Dequeue:** Removes an element from the front of the queue.
3. **Peek:** Displays the front element without removing it.
4. **Display:** Displays all the elements present in the queue.

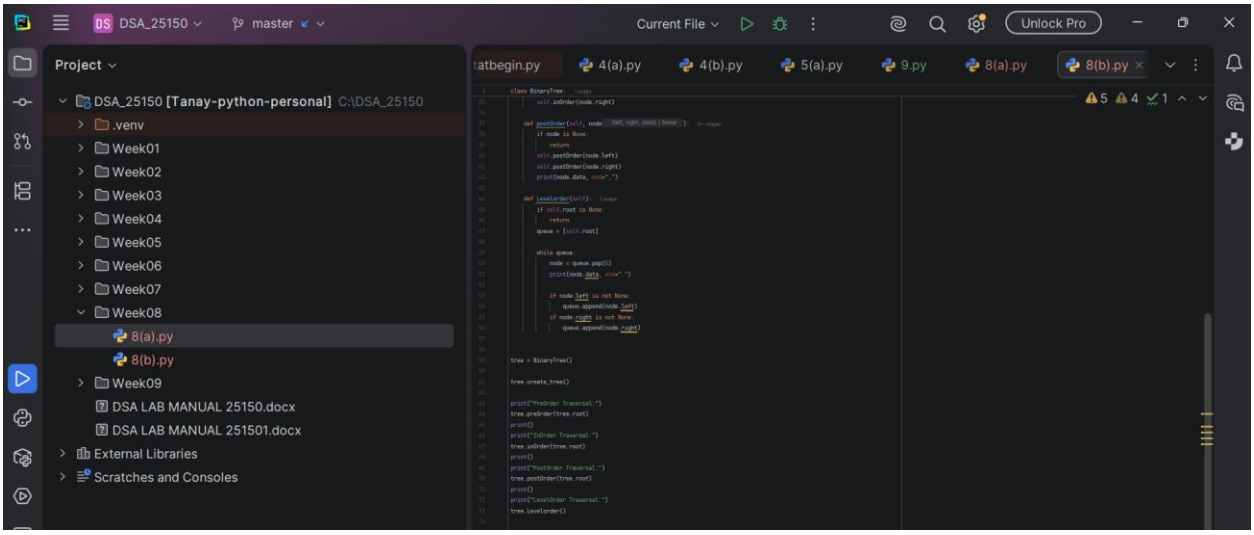
In the array implementation, front and rear indices are used to manage the elements. In the circular queue implementation, the indices wrap around to the beginning of the array when the end is reached, allowing available spaces to be reused. In the singly linked list implementation, each node contains data and a pointer to the next node, while front and rear pointers manage deletion and insertion.

The program checks for overflow and underflow conditions wherever applicable and displays appropriate messages when the queue is full or empty. The menu-driven approach allows the user to perform operations repeatedly until the exit option is selected.

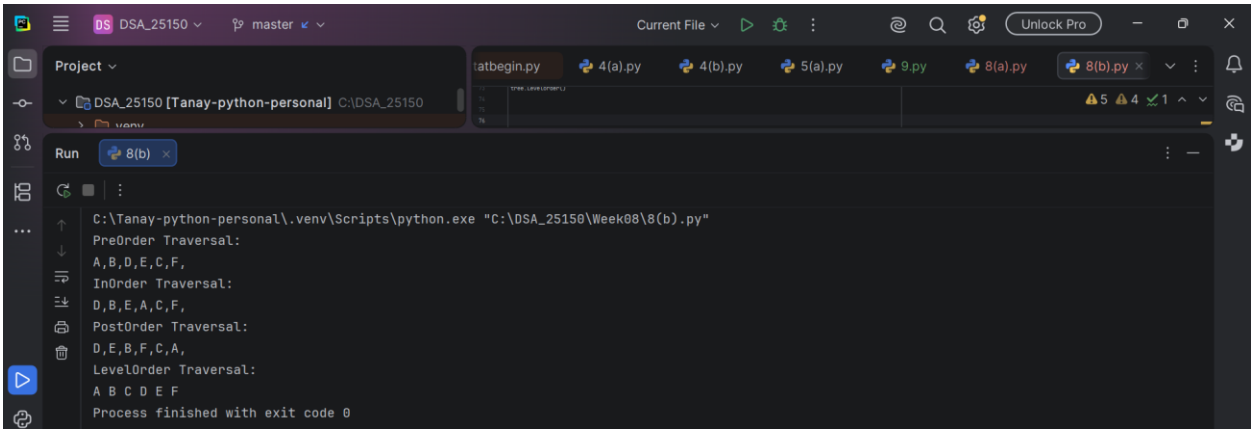
Week-8 Implement Binary Tree with array and with Array and with linked list

(a) With Linked List

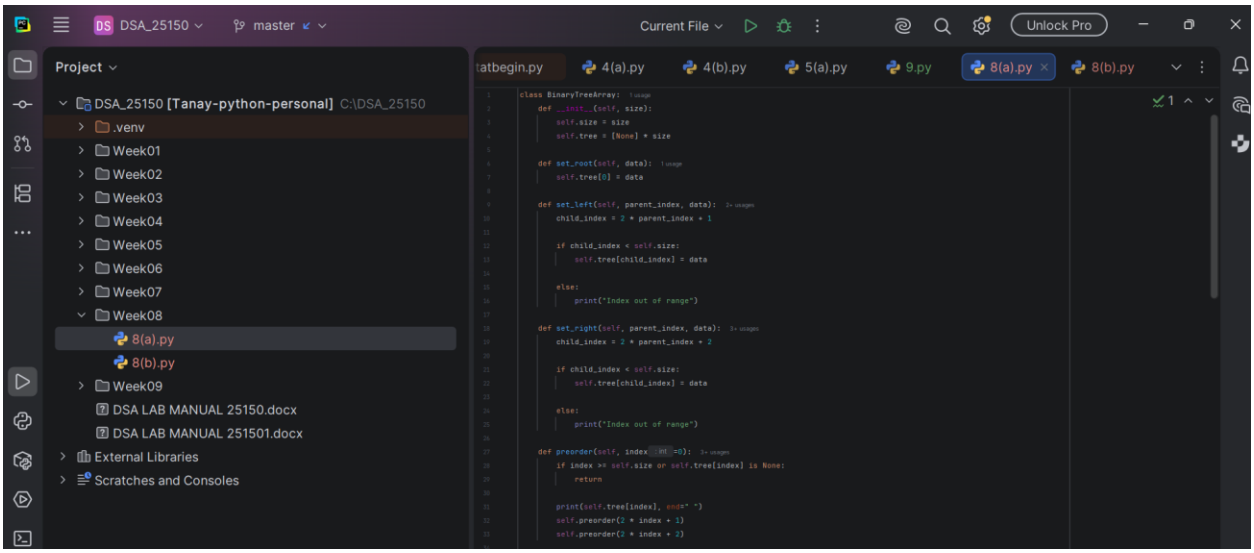


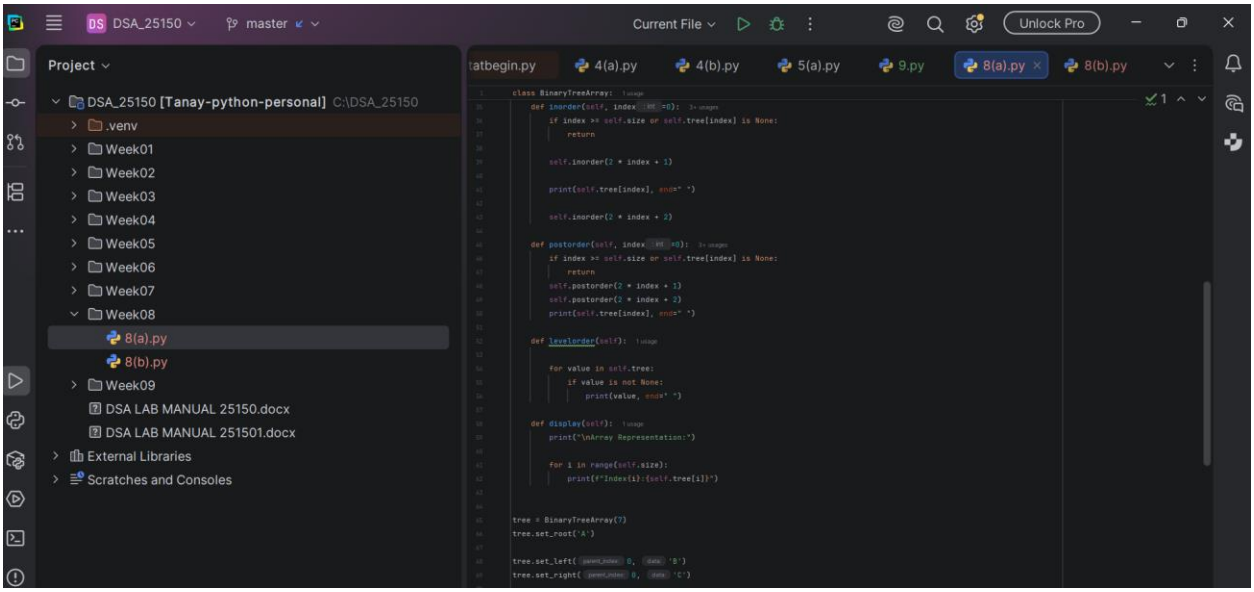


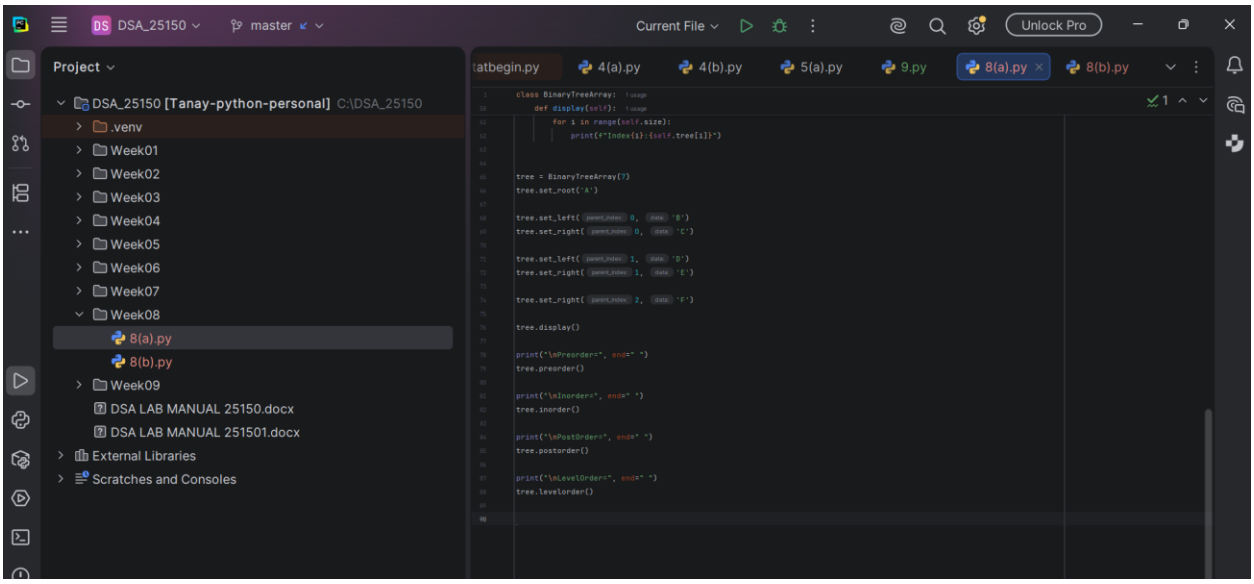
Output:



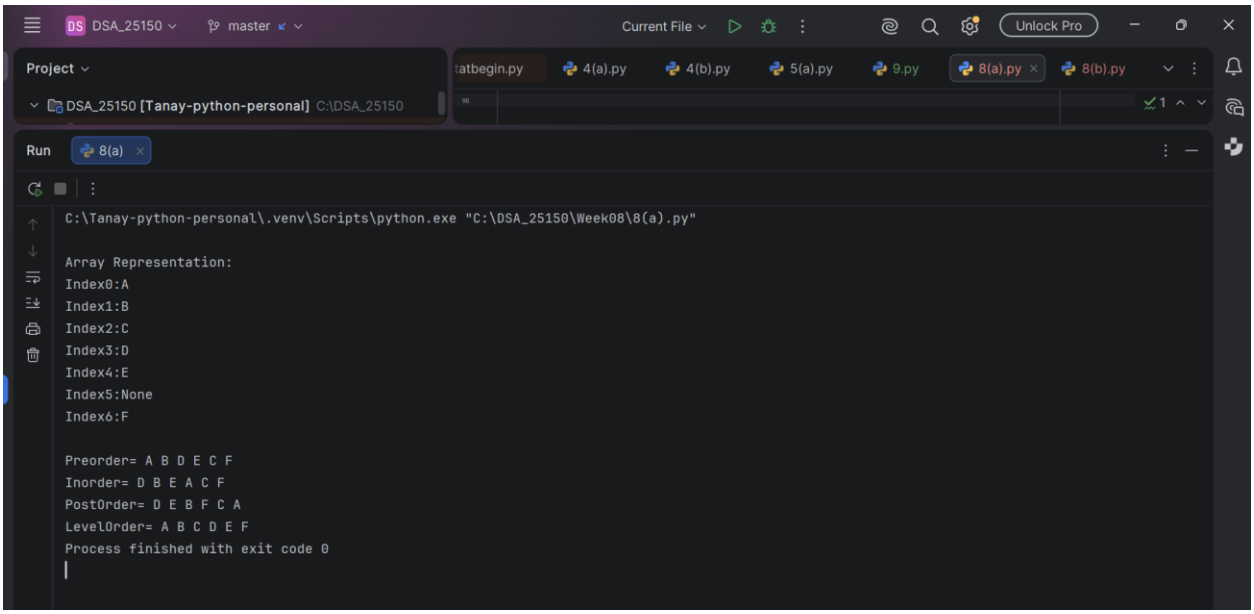
(b) With array


DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
.venv
Week01
Week02
Week03
Week04
Week05
Week06
Week07
Week08
8(a).py
8(b).py
Week09
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
External Libraries
Scratches and Consoles
tatbegin.py
4(a).py
4(b).py
5(a).py
9.py
8(a).py
8(b).py
class BinaryTreeArray:
def __init__(self, size):
self.size = size
self.tree = [None] * size
def set_root(self, data):
self.tree[0] = data
def set_left(self, parent_index, data):
child_index = 2 * parent_index + 1
if child_index < self.size:
self.tree[child_index] = data
else:
print("Index out of range")
def set_right(self, parent_index, data):
child_index = 2 * parent_index + 2
if child_index < self.size:
self.tree[child_index] = data
else:
print("Index out of range")
def preorder(self, index=0):
if index >= self.size or self.tree[index] is None:
return
print(self.tree[index], end=" ")
self.preorder(2 * index + 1)
self.preorder(2 * index + 2)


DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
.venv
Week01
Week02
Week03
Week04
Week05
Week06
Week07
Week08
8(a).py
8(b).py
Week09
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
External Libraries
Scratches and Consoles
tatbegin.py
4(a).py
4(b).py
5(a).py
9.py
8(a).py
8(b).py
def inorder(self, index=0):
if index >= self.size or self.tree[index] is None:
return
self.inorder(2 * index + 1)
print(self.tree[index], end=" ")
self.inorder(2 * index + 2)
def postorder(self, index=0):
if index >= self.size or self.tree[index] is None:
return
self.postorder(2 * index + 1)
self.postorder(2 * index + 2)
print(self.tree[index], end=" ")
def levelorder(self):
for value in self.tree:
if value is not None:
print(value, end=" ")
def display(self):
print("\nArray Representation:")
for i in range(self.size):
print(f"Index{i}:{self.tree[i]}")
tree = BinaryTreeArray(7)
tree.set_root('A')
tree.set_left(0, 'B')
tree.set_right(0, 'C')



Output:



Description

Description:

A Binary Tree is a non-linear data structure in which each node contains data and has at most two children, called the left child and the right child.

This Python program implements a binary tree using a Node class and a BinaryTree class. Each node stores a data element and references to its left and right children. The program creates a binary tree with predefined elements and performs the following traversal operations:

1. Preorder Traversal: Visits the root node, left subtree, and right subtree in that order.
2. Inorder Traversal: Visits the left subtree, root node, and right subtree in that order.
3. Postorder Traversal: Visits the left subtree, right subtree, and root node in that order.
4. Level Order Traversal: Visits the nodes level by level from left to right using a queue.

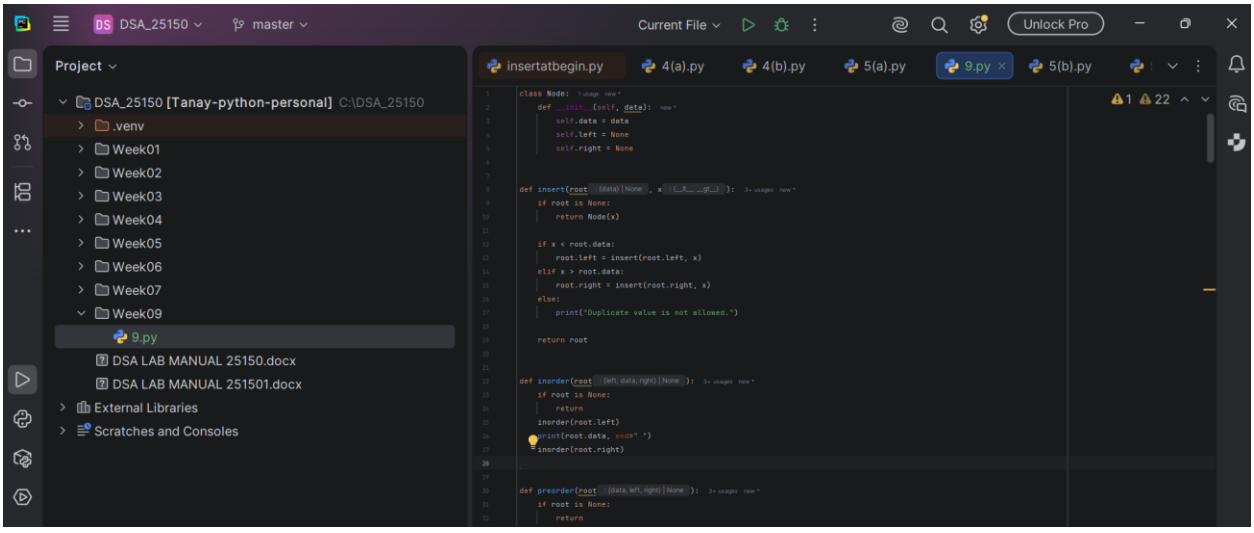
The program creates a binary tree with the elements A, B, C, D, E, and F. The root node is A, B and C are its children, D and E are the children of B, and F is the right child of C.

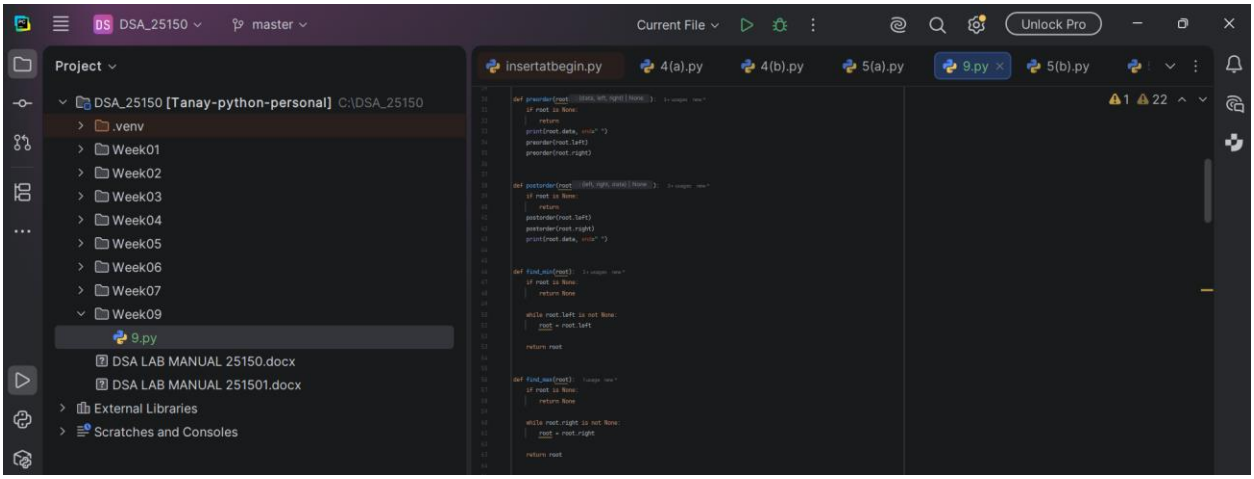
The recursive functions are used to implement preorder, inorder, and postorder traversals. A queue is used in level order traversal to process the nodes level by level.

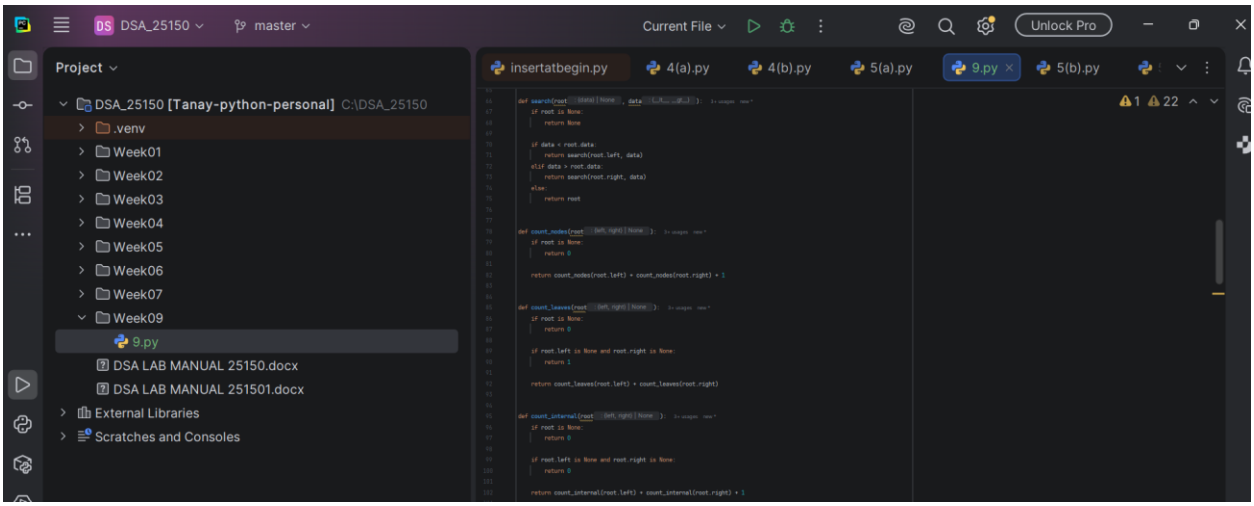
Result: Thus, the binary tree is created successfully, and preorder, inorder, postorder, and level order traversals are performed and displayed using Python.

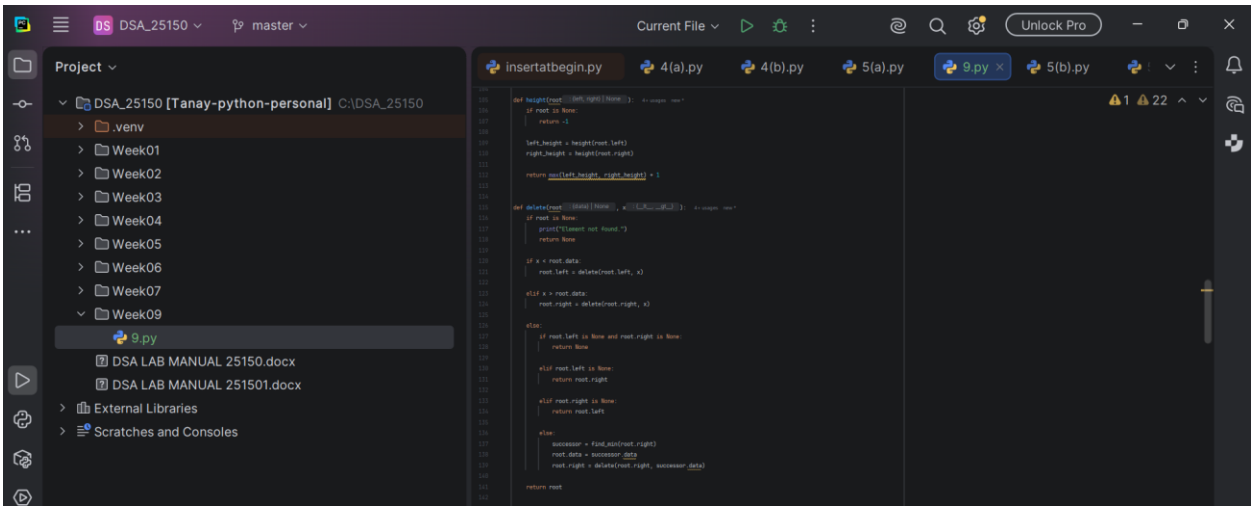
Week-09 Implement BST and its methods

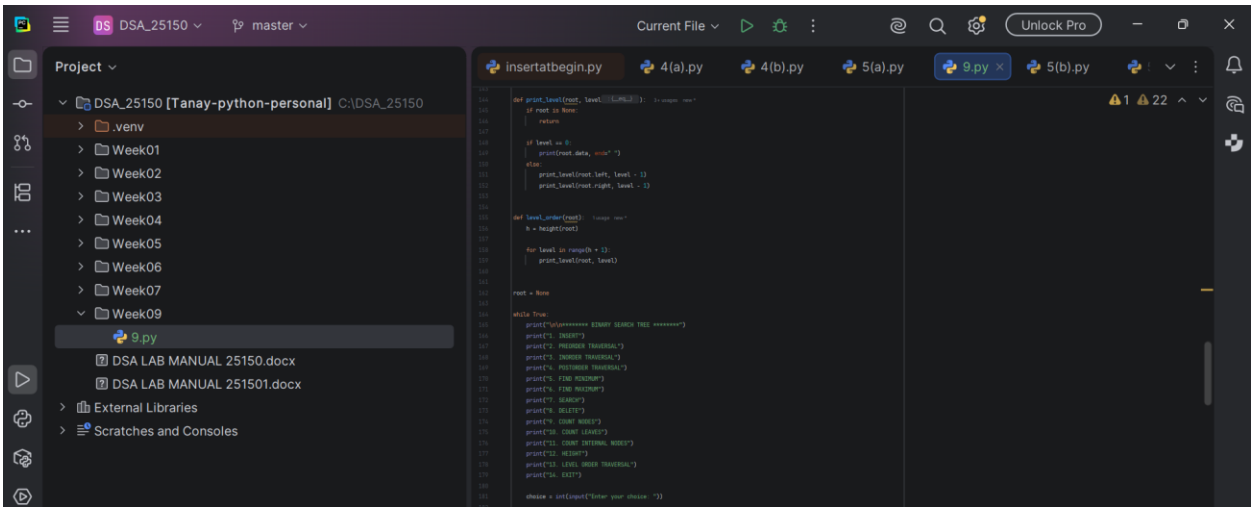
Program:

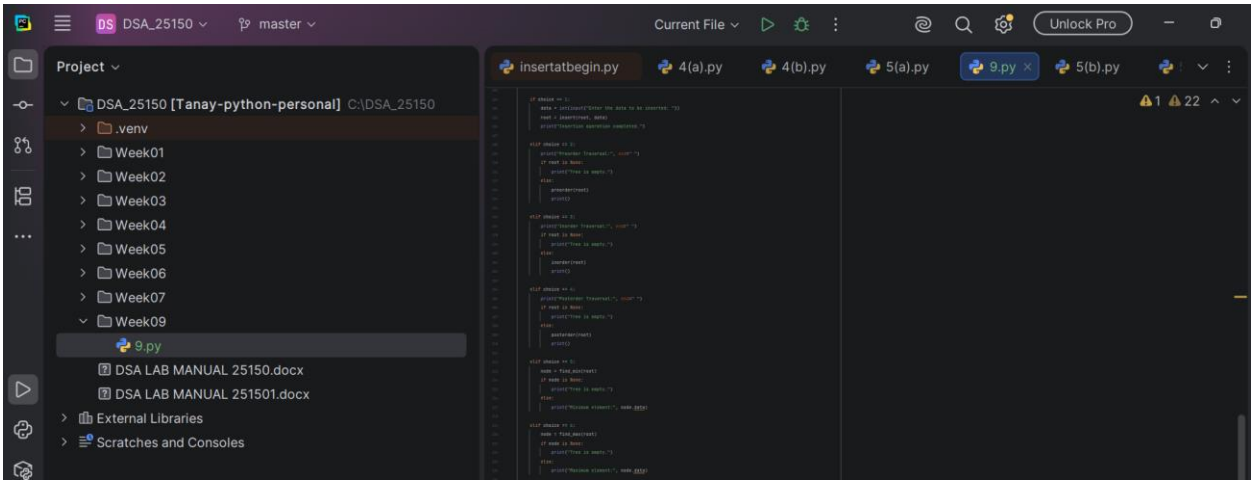


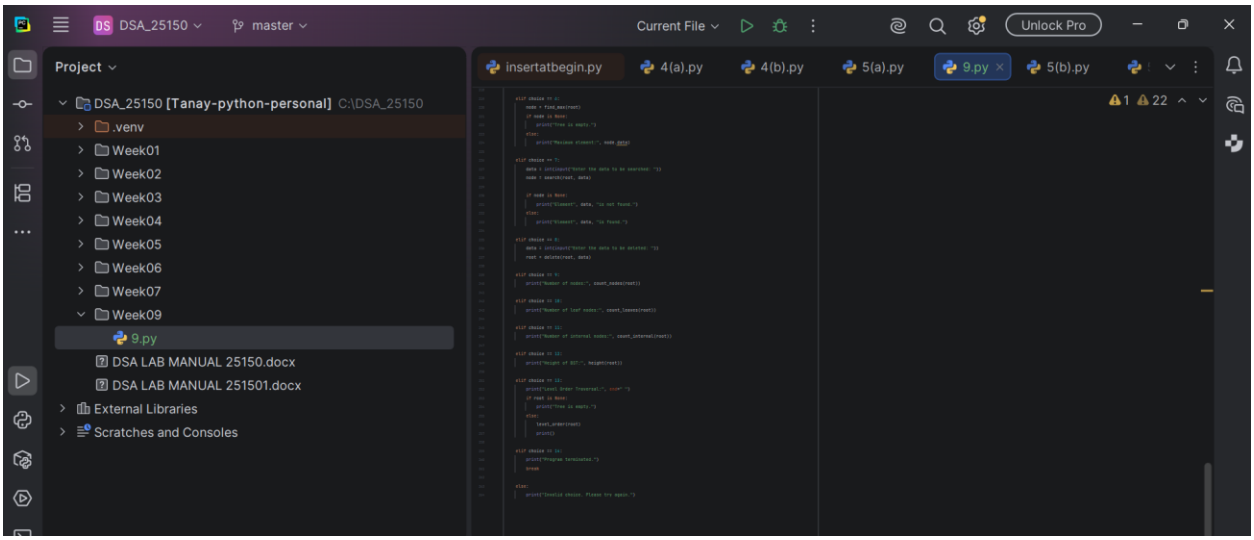

DSA_25150
master
Current File
Unlock Pro
Project
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
DSA_25150 [Tanay-python-personal] C:\DSA_25150
.venv
Week01
Week02
Week03
Week04
Week05
Week06
Week07
Week09
9.py
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
External Libraries
Scratches and Consoles


DSA_25150
master
Current File
Unlock Pro
Project
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
DSA_25150 [Tanay-python-personal] C:\DSA_25150
.venv
Week01
Week02
Week03
Week04
Week05
Week06
Week07
Week09
9.py
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
External Libraries
Scratches and Consoles

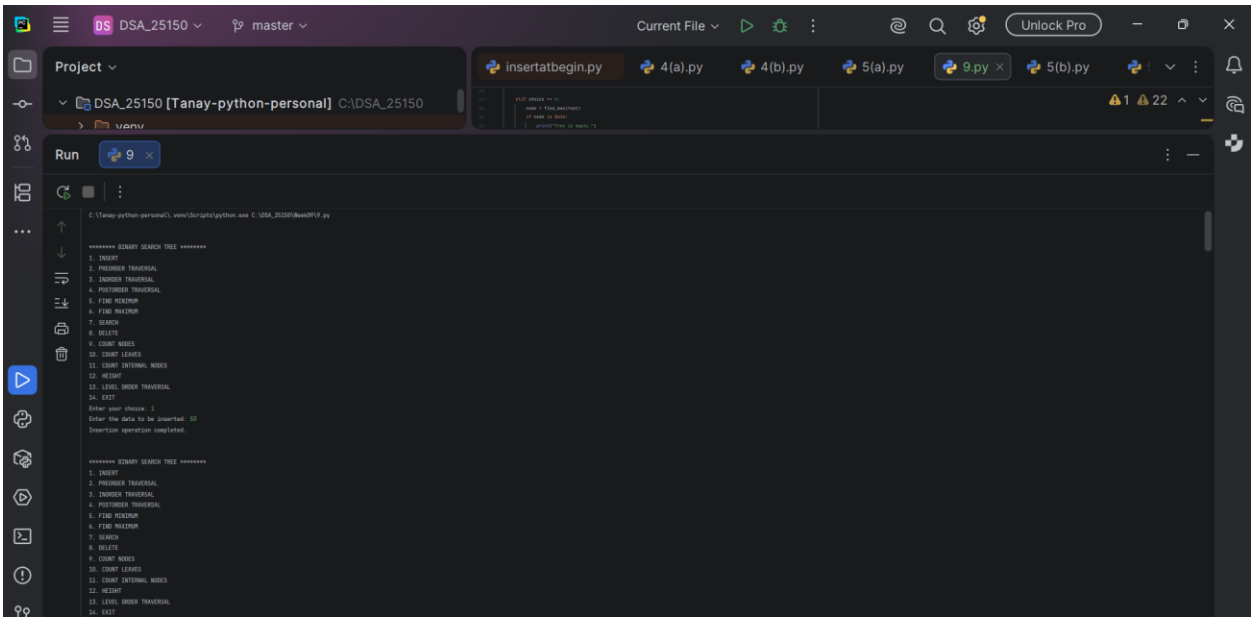

DSA_25150
master
Current File
Unlock Pro
Project
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
DSA_25150 [Tanay-python-personal] C:\DSA_25150
.venv
Week01
Week02
Week03
Week04
Week05
Week06
Week07
Week09
9.py
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
External Libraries
Scratches and Consoles

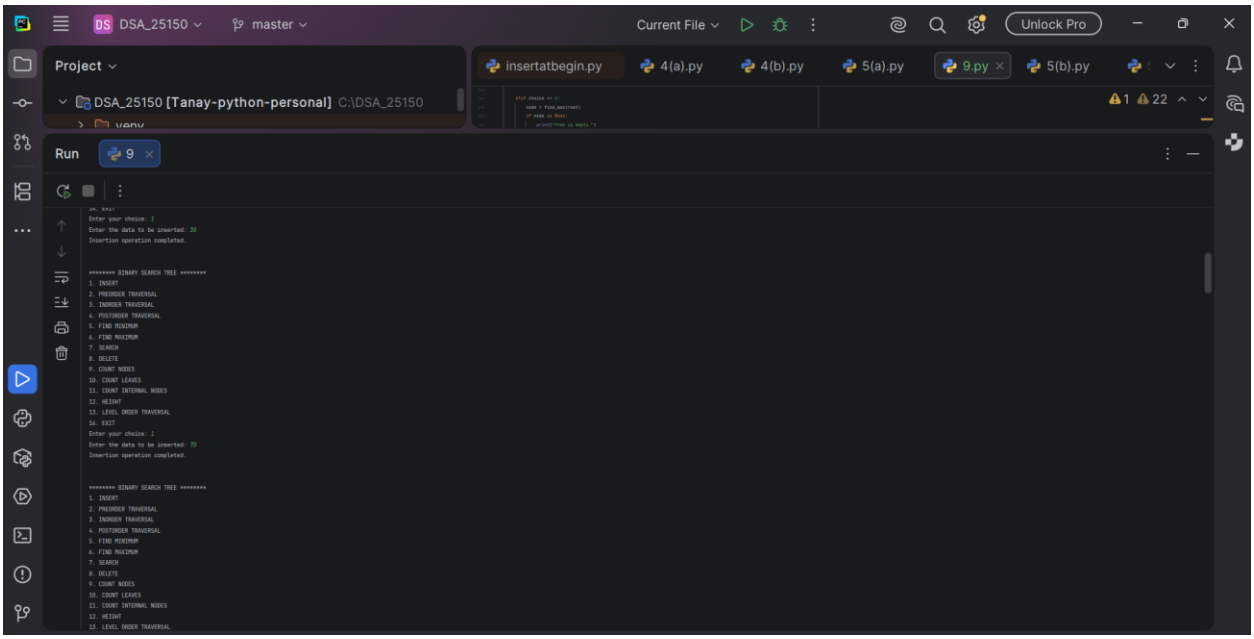

DSA_25150
master
Current File
Unlock Pro
Project
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
DSA_25150 [Tanay-python-personal] C:\DSA_25150
.venv
Week01
Week02
Week03
Week04
Week05
Week06
Week07
Week09
9.py
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
External Libraries
Scratches and Consoles

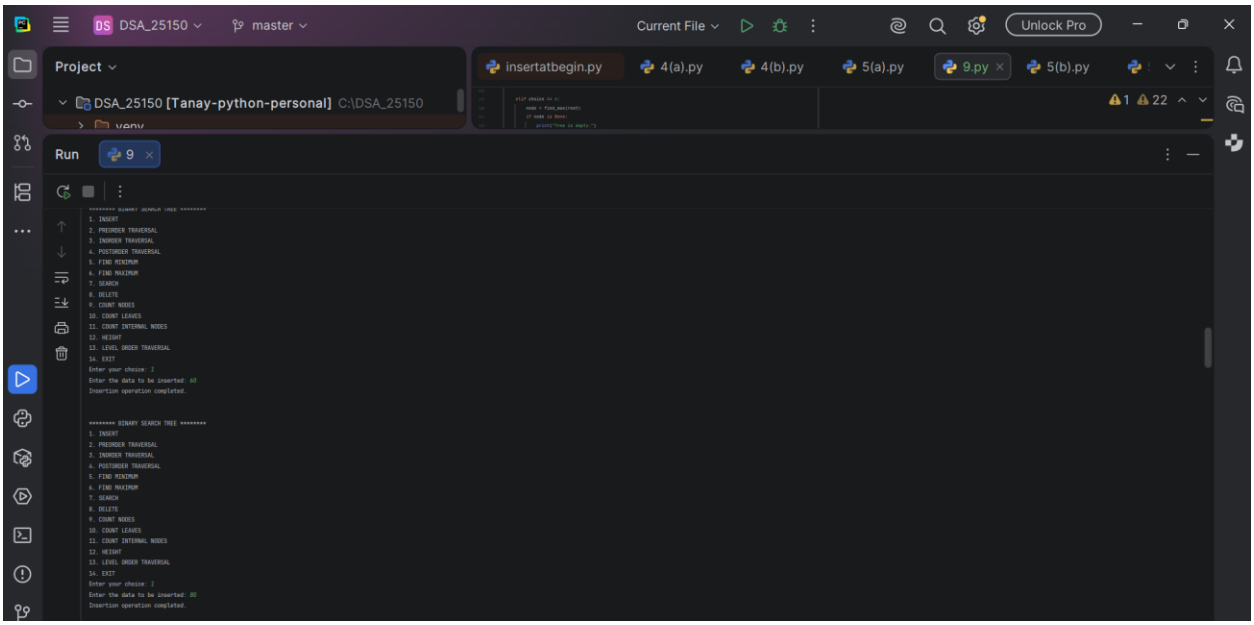

DSA_25150
master
Current File
Unlock Pro
Project
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
DSA_25150 [Tanay-python-personal] C:\DSA_25150
.venv
Week01
Week02
Week03
Week04
Week05
Week06
Week07
Week09
9.py
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
External Libraries
Scratches and Consoles

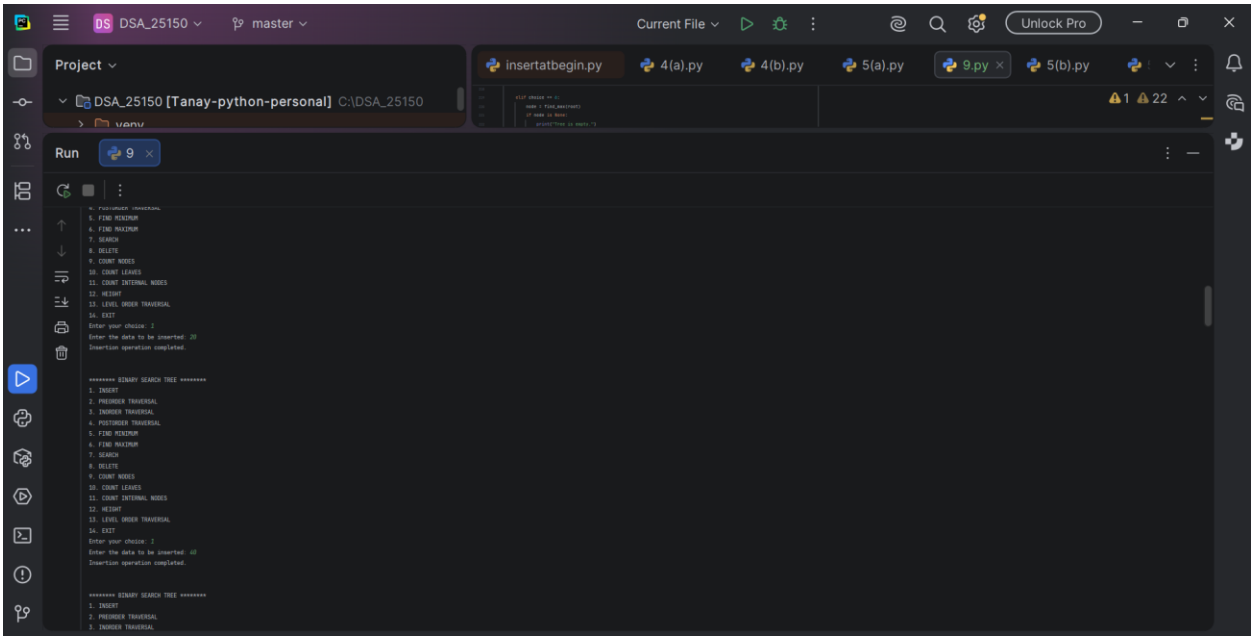

DSA_25150
master
Current File
Unlock Pro
Project
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
DSA_25150 [Tanay-python-personal] C:\DSA_25150
.venv
Week01
Week02
Week03
Week04
Week05
Week06
Week07
Week09
9.py
DSA LAB MANUAL 25150.docx
DSA LAB MANUAL 251501.docx
External Libraries
Scratches and Consoles

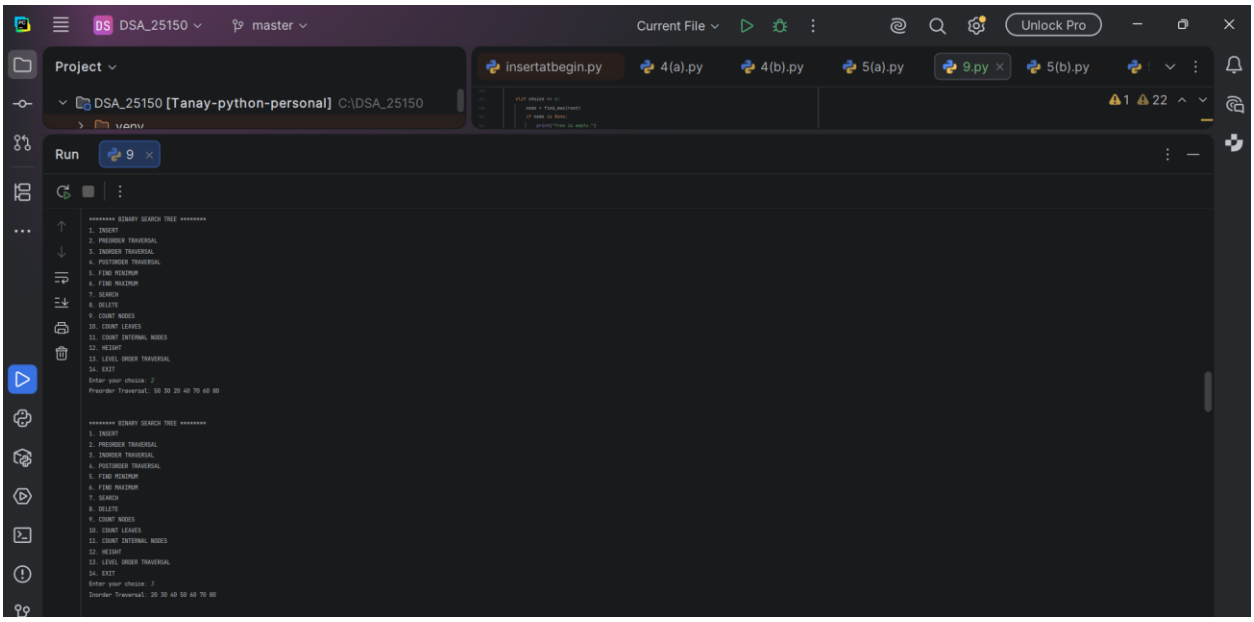
Output:

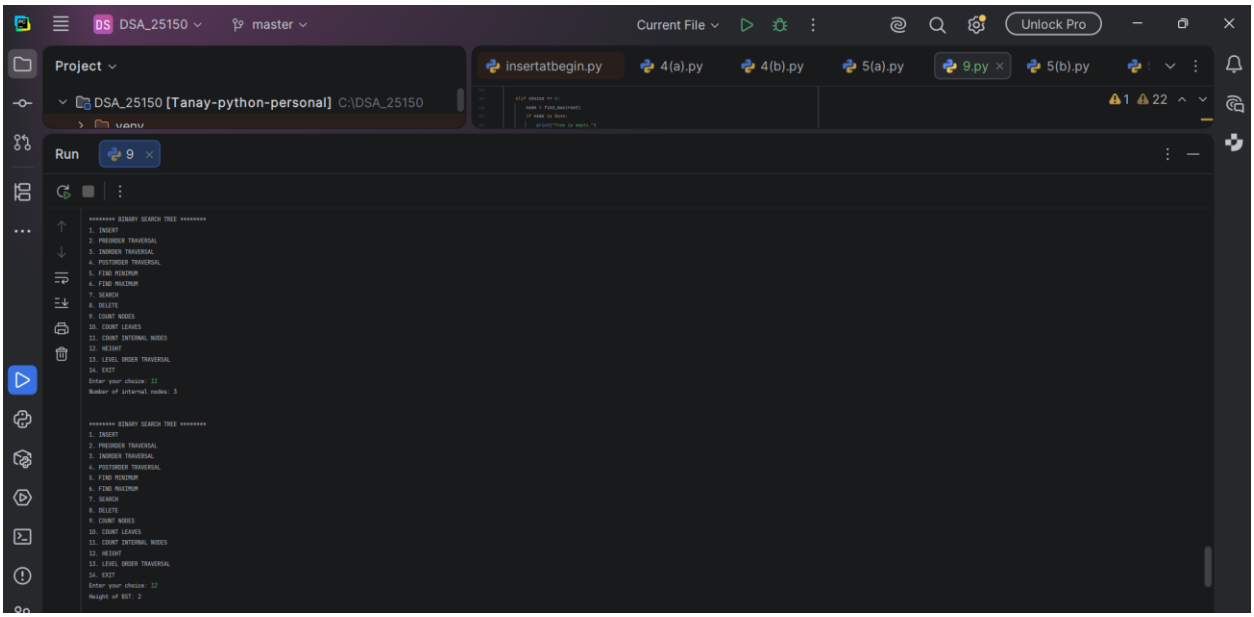

DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
Run
9
C:\Tanay-python-personal\.venv\Scripts\python.exe C:\DSA_25150\Week09\9.py
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 80
Insertion operation completed.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT

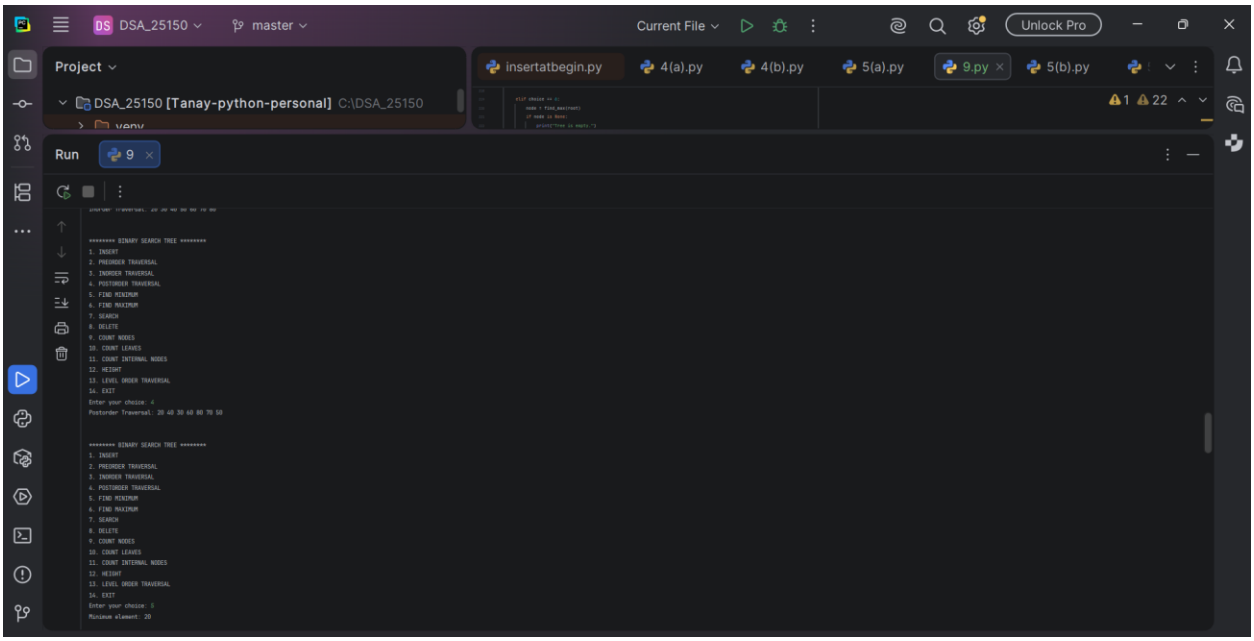

DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
Run
9
Enter your choice: 1
Enter the data to be inserted: 80
Insertion operation completed.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 70
Insertion operation completed.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL

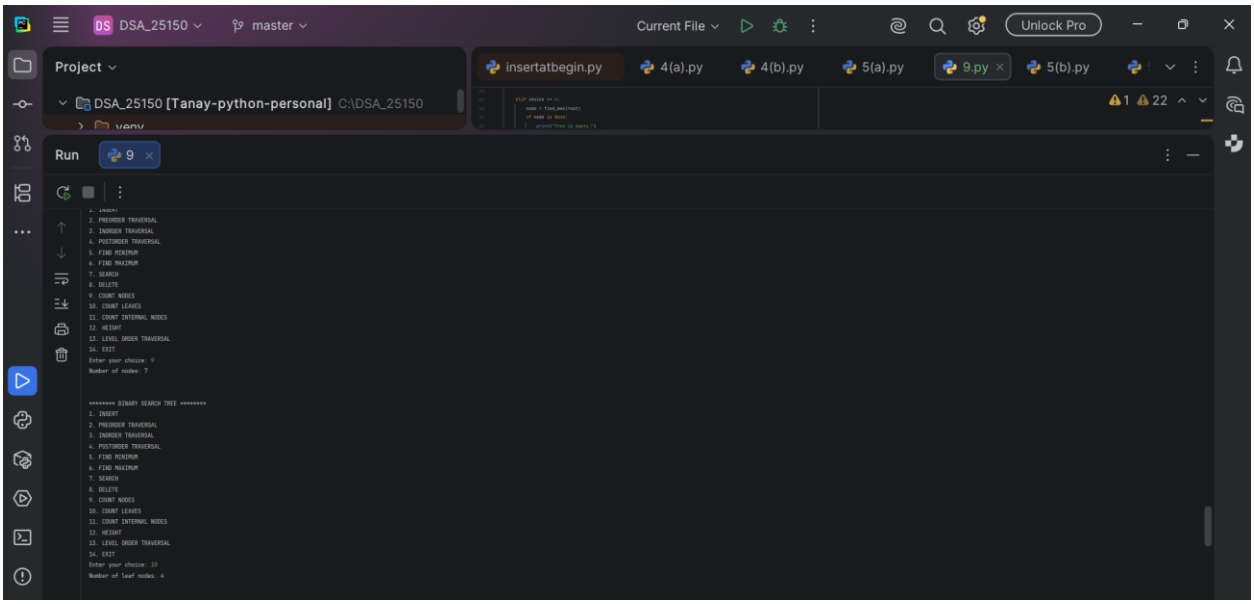

DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
Run
9
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 60
Insertion operation completed.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 80
Insertion operation completed.

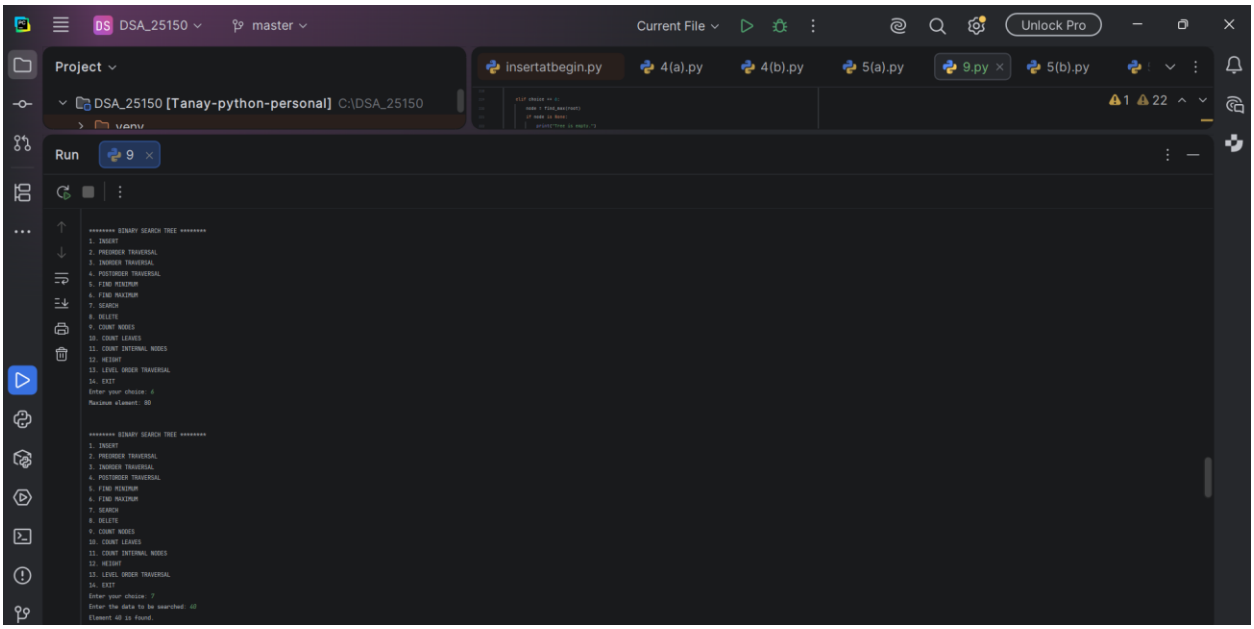

DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
Run
9
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 20
Insertion operation completed.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 60
Insertion operation completed.
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL

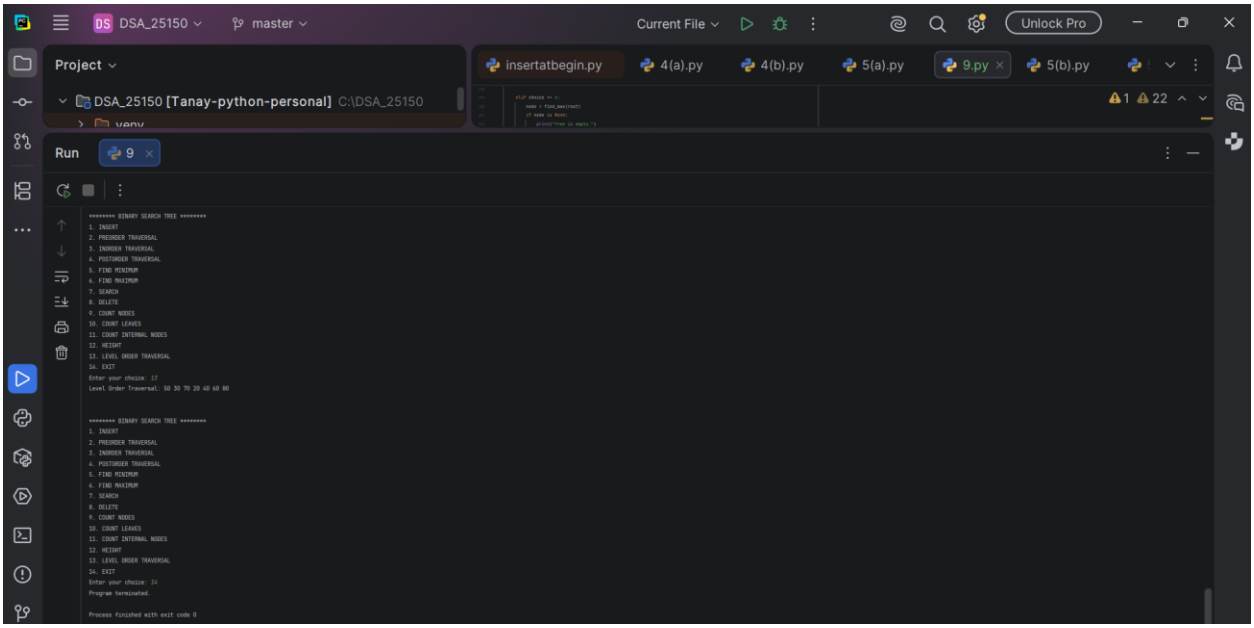

DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
Run
9
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 2
Preorder Traversal: 50 30 20 40 70 60 80
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 3
Inorder Traversal: 20 30 40 50 60 70 80


DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
Run
9
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 11
Number of internal nodes: 3
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 12
Height of BST: 2


DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
Run
9
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 4
Postorder Traversal: 20 40 30 60 80 70 50
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 6
Minimum element: 20


DSA_25150
master
Current File
Unlock Pro
Project
DSA_25150 [Tanay-python-personal] C:\DSA_25150
insertatbegin.py
4(a).py
4(b).py
5(a).py
9.py
5(b).py
Run
9
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 9
Number of nodes: 7
******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 10
Number of leaf nodes: 4





Description:

Binary Search Tree (BST) – Python Program

Description:

A Binary Search Tree (BST) is a binary tree in which each node contains a data value, a left pointer, and a right pointer. The left subtree contains values smaller than the root node, and the right subtree contains values greater than the root node.

This Python program implements a Binary Search Tree using a class and recursive functions. It provides a menu-driven interface to perform the following operations:

1. Insertion: Inserts a new element into the BST while maintaining the BST property. Duplicate values are not allowed.
2. Preorder Traversal: Visits the root node, left subtree, and right subtree.
3. Inorder Traversal: Visits the left subtree, root node, and right subtree. It displays the elements in ascending order.
4. Postorder Traversal: Visits the left subtree, right subtree, and root node.
5. Find Minimum: Finds the minimum element by traversing the leftmost nodes of the tree.
6. Find Maximum: Finds the maximum element by traversing the rightmost nodes of the tree.
7. Search: Searches for a specified element in the BST using comparisons.
8. Deletion: Deletes an element from the BST while maintaining its structure. It handles nodes with no children, one child, or two children.
9. Count Nodes: Calculates the total number of nodes in the tree.
10. Count Leaf Nodes: Counts the nodes that do not have any children.
11. Count Internal Nodes: Counts the nodes that have at least one child, excluding leaf nodes.
12. Find Height: Calculates the height of the BST, defined as the number of edges in the longest path from the root to a leaf. An empty tree has a height of -1.
13. Level Order Traversal: Displays the nodes level by level, starting from the root.
14. Exit: Terminates the program.

The program uses recursion for insertion, searching, deletion, counting, and tree traversals. It continues displaying the menu until the user chooses the exit option.

Result: Thus, the Binary Search Tree is implemented successfully in Python, supporting insertion, deletion, searching, traversal, counting, and height calculation operations.